5

Modular algorithms and interpolation

An important general concept in computer algebra is the idea of using various types of representation for the objects at hand. As an example, we can represent a polynomial either by a list of its coefficients or by its values at sufficiently many points. In fact, this is just computer algebra lingo for the ubiquitous quest for efficient data structures for computational problems.

One successful instantiation of the general concept are **modular algorithms**, where instead of solving an integer problem (more generally, an algebraic computation problem over a Euclidean domain R) directly one solves it modulo one or several integers m. The general principle is illustrated in Figure 5.1. There are three variants: **big prime** (Figure 5.1 with $m = p$ for a prime p), **small primes** (Figure 5.2 with $m = p_1 \cdots p_r$ for pairwise distinct primes $p_1, \ldots, p_r$), and **prime power** modular algorithms (Figure 5.3 with $m = p^l$ for a prime p). The first one is conceptually the simplest, and the basic issues are most visible in that variant. However, the other two variants are computationally superior.

In each case, two technical problems have to be addressed:

- a bound on the solution in R,
- how to find the required moduli.

The first item is sometimes easy, especially when $R = F[x]$. The second item requires the **prime number theorem** when $R = \mathbb{Z}$; for $R = F[x]$ it is easy for the small primes version, but more involved for big primes.

The small primes modular approach has the following advantages over direct computation; the first two also apply to big prime and prime power modular algorithms.

- In some applications, such as computing the greatest common divisor of two polynomials with integer coefficients, we encounter the phenomenon of **intermediate expression swell**: the coefficients of intermediate results during the computation can be much bigger than the coefficients of the final result that we are interested in. In Chapter 6, we will see that the gcd of two polynomials in

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

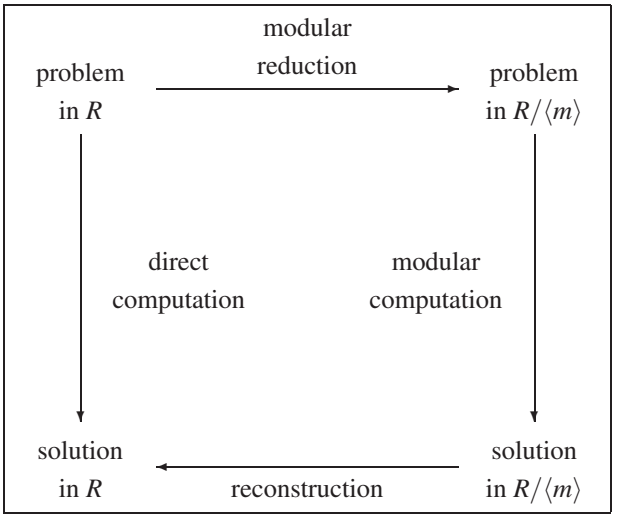


FIGURE 5.1: General scheme for modular algorithms.

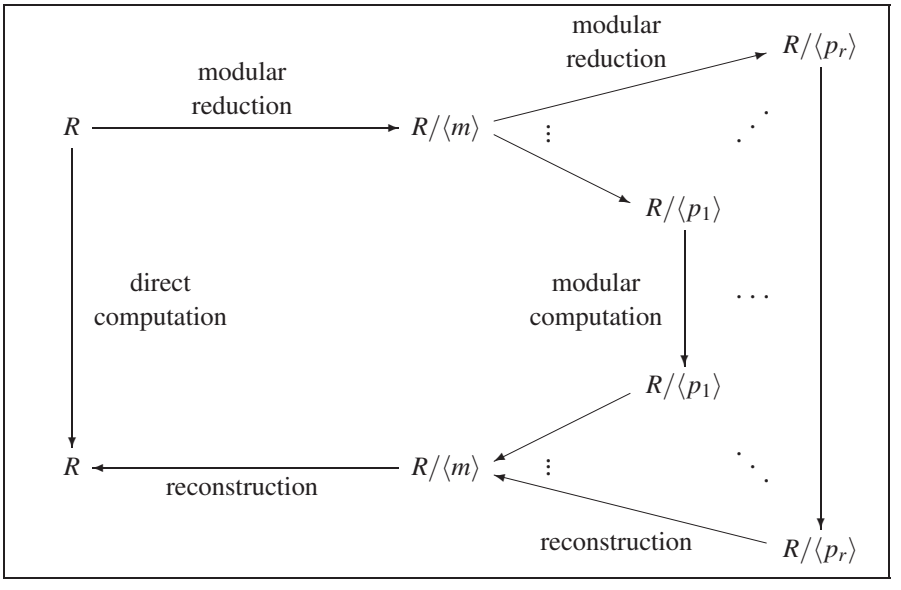


FIGURE 5.2: General scheme for small primes modular algorithms.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

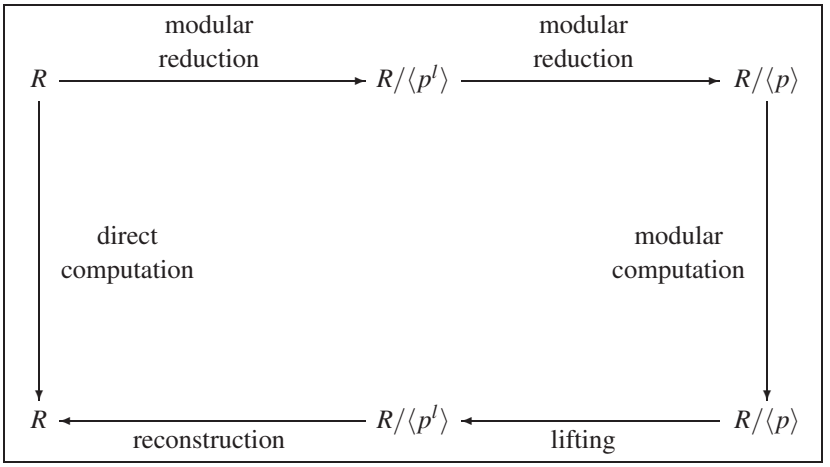


FIGURE 5.3: General scheme for prime power modular algorithms.

$\mathbb{Z}[x]$ of degree at most n has coefficients not much larger than the input polynomials, but the intermediate coefficients in Euclid's algorithm may be longer by a factor of about n than the inputs, and by a factor of about n^2 for the traditional Euclidean Algorithm. When computing in a modular fashion, we may choose the moduli m_i such that their product is only slightly larger than the final result, and by reducing modulo the m_i where possible, also the intermediate results in the modular computation remain as "small" as the final result.

- The designer of a modular algorithm is free in her choice of the moduli, as long as their product is large enough to recover the result. Thus she may choose the moduli to be **Fourier primes** which support particularly fast polynomial arithmetic; these will be discussed in Chapter 8.
- In nearly all tasks in computer algebra, the cost for solving a problem with input size n is at least linear in n. For example, if we use the algorithms for integer and polynomial arithmetic as described in Chapters 2 and 3, then one arithmetic operation on integers of length n or on polynomials of degree n takes $O(n^2)$ operations. In such cases, it is cheaper to solve r "small" problems with inputs of size about n/r rather than one "big" problem. In the extreme case $n = r$, the cost of the modular computation becomes just $O(n)$, but this has to be balanced against the cost for the change of representation.
- If the moduli m_i in the small primes approach fit into one machine word of the target processor, then the cost for an arithmetic operation modulo one m_i amounts to only a few machine cycles.
- The r subtasks modulo the distinct small primes are independent of each other and can be performed in a distributed fashion using r processors or machines in parallel.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

As an aside, we note that as long as the direct computation uses only additions and multiplications, but no divisions, an arbitrary modulus—or arbitrary pairwise coprime moduli in the "small primes" variant—may be chosen in a modular algorithm.

Besides the big prime method, we discuss in this chapter the theoretical underpinnings of the small primes modular algorithm, namely the **Chinese Remainder Algorithm**, and two applications: secret sharing and computing the determinant. We have to wait until Chapter 9 for the tools used in the third variant: Newton iteration and Hensel lifting. The prime power approach will play a major role in the polynomial factorization algorithm in Chapter 15. Table 15.5 on page 460 lists eleven problems for which we will have learnt modular algorithms by then.

We also discuss applications of the Extended Euclidean Algorithm and the Chinese Remainder Algorithm to various kinds of interpolation problems and to partial fraction decomposition.

5.1. Change of representation

There are two basically different types of representation for our objects. In the first, one chooses a **base** and presents data in an expansion along powers of that base. Examples for this type include the coefficient representation of polynomials (where x is the base), more generally a Taylor expansion around u (where $x-u$ is the base; see Section 5.6), and the decimal or binary representation of integers (with base 10 or 2, respectively). Even the usual floating point representation of real numbers can be considered to be in this category, with base 10^{-1}. These types of representation are the "natural" ones, and computer users like to have their input and output in this format.

Prototypical for the second type are the representations of a polynomial by its values at various points $u_0,\ldots,u_{n-1}$, or of an integer modulo different primes $p_0,\ldots,p_{n-1}$. (We number from 0 through $n-1$ instead of 1 through n for consistency with Chapter 10.) The "bases" here are $x-u_0,\ldots,x-u_{n-1}$ and $p_0,\ldots,p_{n-1}$, respectively. Actually, a better analogy is to take

$$\frac{m}{x-u_0},\frac{m}{x-u_1},\ldots,\frac{m}{x-u_{n-1}},$$

where $m=(x-u_0)(x-u_1)\cdots(x-u_{n-1})$, as basis for all polynomials of degree less than n in the first case; see the Lagrange interpolation formula (3) below. Some problems, such as multiplication, are quite easy in appropriate bases of this kind, while others, like division with remainder, seem to require a representation of the first type.

For each computational problem, one should examine whether this general-purpose tool is of use. This involves two questions:

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

- In which representation is the problem easiest to solve?
- How do we convert to and from that representation?

We discuss some fundamental tools for these questions in this book: the Chinese Remainder Algorithm, the Extended Euclidean Algorithm, and Newton iteration (including Hensel lifting).

It is important to realize the similarity between evaluating a polynomial at a point u and taking the remainder of an integer modulo a prime p. The former is the same as taking the remainder modulo $x-u$, and so the latter can be thought of as "evaluating the integer at p". The inverse operation of recovering the coefficients of a polynomial from its values at several points is **interpolation**. For integers, this is afforded by the Chinese Remainder Algorithm, and it is useful to understand this as "interpolating an integer from its values at several primes".

Similar representations exist for rational functions and rational numbers. We will later discuss conversion algorithms: Cauchy interpolation and Padé approximation for rational functions (Sections 5.8 and 5.9), and rational number reconstruction (Section 5.10).

The proper choice of representation is vital when dealing with multivariate polynomials. Four important possibilities—dense, sparse, by an arithmetic circuit, or by a "black box"—are briefly discussed in Section 16.6.

An important application of the general idea of change of representation are the fast multiplication algorithms in Chapter 8 (based on the FFT). In the three major problems in Part V, namely Gröbner bases, integration, and summation, the basic task can be interpreted as transforming a general input into a representation where the problem at hand is fairly easy to solve.

5.2. Evaluation and interpolation

We start with the simplest but most important change of representation: evaluation and interpolation. Suppose that F is a field and $u_0,\ldots,u_{n-1}\in F$ are pairwise distinct. A polynomial $f=\sum_{0\le i<n} f_i x^i \in F[x]$ can be evaluated at a single point $u\in F$ with $n-1$ multiplications and $n-1$ additions in F, using **Horner's rule**

$$f(u)=(\cdots(f_{n-1}u+f_{n-2})u+\cdots+f_1)u+f_0. \tag{1}$$

Thus we can evaluate F at all points u_i using $2n^2-2n$ operations in F. What about interpolation?

The **Lagrange interpolant**

$$l_i=\prod_{\substack{0\le j<n\\ j\ne i}}\frac{x-u_j}{u_i-u_j}\in F[x] \tag{2}$$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

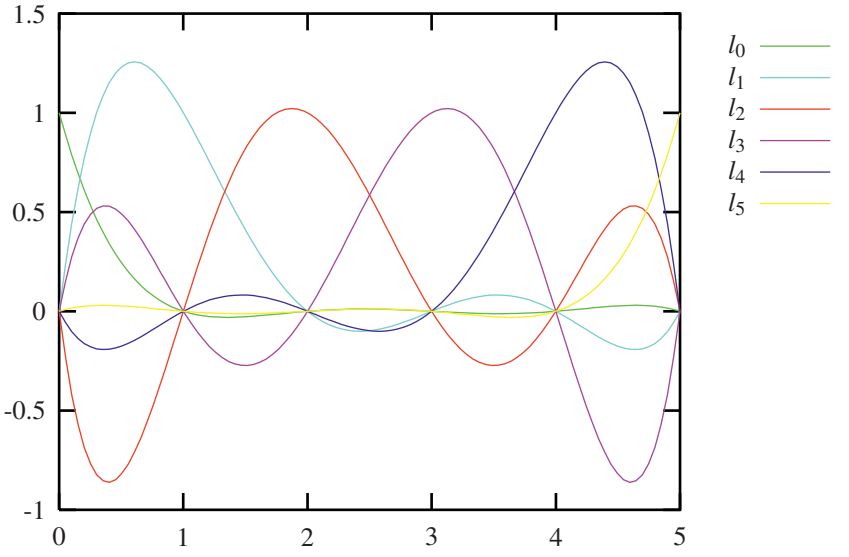


FIGURE 5.4: The Lagrange interpolants $l_0, \ldots, l_5$ when $u_i = i$ for $i = 0, \ldots, 5$.

has the property that $l_i(u_j)$ is 0 if $i \neq j$ and 1 when $i = j$; see Figure 5.4. For arbitrary $v_0, \ldots, v_{n-1} \in F$,

$$f = \sum_{0 \leq i < n} v_i l_i = \sum_{0 \leq i < n} v_i \prod_{\substack{0 \leq j < n \\ j \neq i}} \frac{x - u_j}{u_i - u_j}. \tag{3}$$

is a polynomial of degree less than n such that $f(u_i) = v_i$ for all i. The interpolating polynomial with this degree constraint is unique, since the difference of two such polynomials has degree less than n and n roots, hence is the zero polynomial.

THEOREM 5.1.
Evaluating a polynomial $f \in F[x]$ of degree less than n at n distinct points $u_0, \ldots, u_{n-1} \in F$ or computing an interpolating polynomial at these points can be performed with $O(n^2)$ operations in F. More precisely, evaluation takes $2n^2 - 2n$ operations, and Lagrange interpolation uses $7n^2 - 7ny$ operations.

PROOF. It remains to prove the claim for interpolation. Let $m_i = x - u_i$ for all i. We first compute $m_0 m_1, m_0 m_1 m_2, \ldots, m = m_0 \cdots m_{n-1}$. This amounts to multiplying a monic linear polynomial by a monic polynomial of degree i for $1 \leq i < n$, taking

$$\sum_{1 \leq i < n} 2i = n^2 - n$$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

arithmetic operations. Then for each i, we divide m by m_i, taking $2n-2$ operations (Exercise 5.3), evaluate m/m_i at u_i, taking at most $2n-3$ operations since m/m_i is monic, and divide v_i by that value. This amounts to $4n^2-4n$ operations for all i. Finally, computing the linear combination (3) takes another $2n^2-2n$ operations, and the estimate follows by adding up. □

Exercise 5.11 discusses a different algorithm, **Newton interpolation**, and shows that it takes at most $\frac{5}{2}n^2$ field operations. In Chapter 10, we show that both evaluation and interpolation can be done much faster, with only $O(n\log^2 n\log\log n)$ operations.

A global view of evaluation and interpolation is the following. Given n points $u_0,\ldots,u_{n-1}\in F$, we define the evaluation map $\chi\colon F^n\longrightarrow F^n$ by

$$\chi(f_0,\ldots,f_{n-1})=\Big(\sum_{0\le j<n} f_j u_0^j,\ldots,\sum_{0\le j<n} f_j u_{n-1}^j\Big) \tag{4}$$

This is just the map corresponding to evaluation of polynomials $f=\sum f_i x^i$ of degree less than n at $u_0,\ldots,u_{n-1}$. We consider χ as a function of the coefficients f_i; different evaluation points u_i give different functions. The map χ is obviously F-linear and represented by the **Vandermonde matrix**

$$V=\mathrm{VDM}(u_0,\ldots,u_{n-1})=\begin{pmatrix} 1 & u_0 & u_0^2 & \cdots & u_0^{n-1}\\ 1 & u_1 & u_1^2 & \cdots & u_1^{n-1}\\ 1 & u_2 & u_2^2 & \cdots & u_2^{n-1}\\ \vdots & \vdots & \vdots & & \vdots\\ 1 & u_{n-1} & u_{n-1}^2 & \cdots & u_{n-1}^{n-1}\end{pmatrix}\in F^{n\times n}.$$

If $u_i=u_j$ for some $i\neq j$, then row i and row j of V are equal, and V is singular. On the other hand, if all u_i are distinct, then the existence of a unique interpolation polynomial of degree less than n shows that χ (and hence V) is invertible, and then V^{-1} is the matrix of the interpolation map. Both evaluation and interpolation are linear maps between coefficient and value vectors, but of course not linear in $u_0,\ldots,u_{n-1}$.

5.3. Application: Secret sharing

A neat application of interpolation, which we mentioned in Section 1.3 but which will not be used later, is secret sharing: you want to give to n players a shared secret, so that together they can discover it, but no proper subset of the players can. To achieve this, you identify possible secrets with elements of the finite field $\mathbb{F}_p=\mathbb{Z}/\langle p\rangle$ for an appropriate p. Some bank cards for Automatic Teller Machine access have as their secret PIN codes four-digit decimal numbers. For such a secret, you choose a prime p just bigger than 10000, say $p=10007$. Then you

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

choose $2n-1$ random elements $f_1,\dots,f_{n-1},u_0,\dots,u_{n-1}\in\mathbb{F}_p$ uniformly and independently with all u_i nonzero, call your secret f_0, set $f=f_{n-1}x^{n-1}+\cdots+f_1x+f_0\in\mathbb{F}_p[x]$, and give to player number i the value $f(u_i)\in\mathbb{F}_p$. (If $u_i=u_j$ for some $i\neq j$, you have to make a new random choice; this is unlikely to happen if $n\ll\sqrt{p}$.) Then together they can determine the (unique) interpolation polynomial f of degree less than n, and thus f_0. But if any smaller number of them, say $n-1$, get together, then the possible interpolation polynomials consistent with this partial knowledge are such that each value in $\mathbb{F}_p$ of f_0 is equally likely: they have no information on f_0 (Exercise 5.14).

We can extend this scheme to the situation where $k\leq n$ and each subset of k players are able to recover the secret, but no set of fewer than k players can. This is achieved by randomly and independently choosing $n+k-1$ elements $u_0,\dots,u_{n-1},f_1,\dots,f_{k-1}\in\mathbb{F}_p$ and giving $f(u_i)$ to player i, where $f=f_{k-1}x^{k-1}+\cdots+f_1x+f_0\in\mathbb{F}_p[x]$ and $f_0\in\mathbb{F}_p$ is the secret as above. Again, it is required that $u_i\neq u_j$ if $i\neq j$. Since f is uniquely determined by its values at k points, each subset of k out of the n players can calculate f and thus the secret f_0, but fewer than k players together have no information on f_0.

5.4. The Chinese Remainder Algorithm

Suppose that $f\in\mathbb{N}$ has two decimal digits and has remainder 2 on division by 11 and 7 on division by 13. Does this uniquely define f, and if so, is there a better way to find it than to check all values between 0 and 99? We will see in this section that the answer to both questions is positive.

For this section, R is a Euclidean domain, and we fix the following notation:

$$\begin{gathered}m_0,\dots,m_{r-1}\in R \text{ are pairwise coprime, so that } \gcd(m_i,m_j)=1\\ \text{for } 0\leq i<j<r, \text{ and } m=m_0\cdots m_{r-1}.\end{gathered}\tag{5}$$

Thus $m=\operatorname{lcm}(m_0,\dots,m_{r-1})$. For $0\leq i<r$, we have the canonical ring homomorphism

$$\begin{aligned}\pi_i\colon R &\longrightarrow R/\langle m_i\rangle,\\ f &\longmapsto f \bmod m_i.\end{aligned}$$

Combining these for all i, we get the ring homomorphism

$$\begin{aligned}\chi=\pi_0\times\cdots\times\pi_{r-1}\colon R &\longrightarrow R/\langle m_0\rangle\times\ldots\times R/\langle m_{r-1}\rangle,\\ f &\longmapsto (f \bmod m_0,\dots,f \bmod m_{r-1}).\end{aligned}$$

For our example above, we have $R=\mathbb{Z}$, $r=2$, $m_0=11$, $m_1=13$, $m=143$, and

$$\chi(f)=(f \bmod 11, f \bmod 13)=(2 \bmod 11, 7 \bmod 13)\in\mathbb{Z}_{11}\times\mathbb{Z}_{13}.$$

The following statement provides, in somewhat abstract terminology, the theoretical basis for many of our algorithms.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

THEOREM 5.2.
χ is surjective with kernel $\langle m\rangle$.

PROOF. Let $f \in R$. Then

$$\begin{aligned} f \in \ker\chi &\iff \chi(f) = (f \bmod m_0, \ldots, f \bmod m_{r-1}) = (0,\ldots,0) \\ &\iff m_i \mid f \text{ for } 0 \le i < r \iff \operatorname{lcm}(m_0,\ldots,m_{r-1}) \mid f \iff m \mid f, \end{aligned}$$

and $\ker\chi = \langle m\rangle$. For the surjectivity, it is sufficient to show that a "Lagrange interpolant" $l_i \in R$ with $\chi(l_i) = e_i$ exists for $0 \le i < r$, where $e_i = (0,\ldots,0,1,0,\ldots,0) \in R/\langle m_0\rangle \times \cdots \times R/\langle m_{r-1}\rangle$ denotes the ith unit vector. To see why this is enough, let

$$v = (v_0 \bmod m_0, \ldots, v_{r-1} \bmod m_{r-1}) \in R/\langle m_0\rangle \times \cdots \times R/\langle m_{r-1}\rangle$$

be arbitrary, with $v_0,\ldots,v_{r-1} \in R$. Then

$$\begin{aligned} \chi\left(\sum_{0\le i<r} v_i l_i\right) &= \sum_{0\le i<r} \chi(v_i)\chi(l_i) = \sum_{0\le i<r} (v_i \bmod m_0, \ldots, v_i \bmod m_{r-1}) \cdot e_i \\ &= \sum_{0\le i<r} (0,\ldots,0,v_i \bmod m_i,0,\ldots,0) = v. \end{aligned}$$

We assume $i = 0$ for simplicity. The Extended Euclidean Algorithm, when applied to $m_1 \cdots m_{r-1} = m/m_0$ and m_0, computes $s,t \in R$ with $sm/m_0 + tm_0 = 1 = \gcd(m/m_0, m_0)$. If we now let $l_0 = sm/m_0$, then obviously $l_0 \equiv 0 \bmod m_j$ for $1 \le j < r$, and

$$l_0 = s\frac{m}{m_0} \equiv s\frac{m}{m_0} + tm_0 = 1 \bmod m_0,$$

so that $\chi(l_0) = e_0$ as claimed. □

COROLLARY 5.3 *Chinese Remainder Theorem (CRT).*
We have the ring isomorphism

$$R/\langle m\rangle \cong R/\langle m_0\rangle \times \cdots \times R/\langle m_{r-1}\rangle \tag{6}$$

and the group isomorphism of the multiplicative groups

$$(R/\langle m\rangle)^\times \cong (R/\langle m_0\rangle)^\times \times \cdots \times (R/\langle m_{r-1}\rangle)^\times.$$

PROOF. Theorem 5.2 and the homomorphism theorem for rings (Section 25.2) imply (6). For $f \in R$, we have

$$\begin{aligned} f \text{ is invertible modulo } m &\iff \gcd(f,m) = 1 \iff \gcd(f,m_i) = 1 \text{ for } 0 \le i < r \\ &\iff f \text{ is invertible modulo } m_i \text{ for } 0 \le i < r, \end{aligned}$$

and the second claim follows. □

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

The proof of Theorem 5.2 is constructive and yields the following algorithm.

ALGORITHM 5.4 Chinese Remainder Algorithm (CRA).
Input: $m_0, \ldots, m_{r-1} \in R$ pairwise coprime, $v_0, \ldots, v_{r-1} \in R$, where R is a Euclidean domain.
Output: $f \in R$ such that $f \equiv v_i \bmod m_i$ for $0 \leq i < r$.

1. $m \longleftarrow m_0 \cdots m_{r-1}$
2. **for** $i = 0, \ldots, r-1$ **do**
   compute m/m_i
   call the Extended Euclidean Algorithm 3.14 to compute $s_i, t_i \in R$ with $s_i \dfrac{m}{m_i} + t_i m_i = 1$
   $c_i \longleftarrow v_i s_i \text{ rem } m_i$
3. **return** $\displaystyle\sum_{0 \leq i < r} c_i \frac{m}{m_i}$

We recall that c_i is the remainder in R on dividing $v_i s_i$ by m_i (Section 3.1). To see that the algorithm works correctly, we observe that $c_i m/m_i \equiv 0 \bmod m_j$ for $j \neq i$ and $c_i m/m_i \equiv v_i s_i m/m_i \equiv v_i \bmod m_i$, and hence $f \equiv c_i m/m_i \equiv v_i \bmod m_i$ for $0 \leq i < r$, as claimed.

EXAMPLE 5.5. (i) We let $R = \mathbb{Z}$, $m_i = p_i^{e_i}$ for $0 \leq i < r$, where the $p_i \in \mathbb{N}$ are distinct primes and $e_i \in \mathbb{N}_{>0}$ for $0 \leq i < r$. Then

$$m = \prod_{0 \leq i < r} p_i^{e_i}$$

is the prime decomposition of $m \in \mathbb{Z}$. The CRT tells us that

$$\mathbb{Z}/\langle m \rangle \cong \mathbb{Z}/\langle p_0^{e_0} \rangle \times \cdots \times \mathbb{Z}/\langle p_{r-1}^{e_{r-1}} \rangle,$$

and for arbitrary $v_0, \ldots, v_{r-1} \in \mathbb{Z}$ the CRA computes a solution $f \in \mathbb{Z}$ of the system of congruences

$$f \equiv v_i \bmod p_i^{e_i} \text{ for } 0 \leq i < r.$$

For example, we take $r = 2$, $m_0 = 11$, $m_1 = 13$, and $m = 11 \cdot 13 = 143$, and find for $v_0 = 2$ and $v_1 = 7$ an $f \in \mathbb{Z}$ with $0 \leq f < m$ and

$$f \equiv 2 \bmod 11, \quad f \equiv 7 \bmod 13.$$

There is nothing to do in step 1 of Algorithm 5.4. In step 2, we have to apply the Extended Euclidean Algorithm to 11 and 13 and get $6 \cdot 13 + (-7) \cdot 11 = 1$, that is,

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

$s_0 = 6$ and $s_1 = -7$. The Lagrange interpolants l_0 and l_1 from Theorem 5.2, which do not occur explicitly in the algorithm, are $l_0 = 6 \cdot 13 = 78$ and $l_1 = (-7) \cdot 11 = -77$, and we check that in fact $l_0 \equiv 1 \bmod 11$, $l_0 \equiv 0 \bmod 13$, $l_1 \equiv 0 \bmod 11$, and $l_1 \equiv 1 \bmod 13$. Now

$$\begin{aligned} c_0 &= v_0 s_0 \text{ rem } m_0 = 2 \cdot 6 \text{ rem } 11 = 1, \\ c_1 &= v_1 s_1 \text{ rem } m_1 = 7 \cdot (-7) \text{ rem } 13 = 3. \end{aligned}$$

Finally, in step 3 we compute

$$f = c_0 \frac{m}{m_0} + c_1 \frac{m}{m_1} = 1 \cdot 13 + 3 \cdot 11 = 46,$$

and indeed $46 = 4 \cdot 11 + 2 = 3 \cdot 13 + 7$.

(ii) We let $R = F[x]$ for a field F and $m_i = x - u_i$ for $0 \leq i < r$, where $u_0, \ldots, u_{r-1}$ in F are pairwise distinct. Then $f \equiv f(u_i) \bmod (x - u_i)$ for $0 \leq i < r$ and arbitrary $f \in F[x]$, by Section 4.1, and hence the ring homomorphism

$$\begin{array}{rcl} \chi : F[x] & \longrightarrow & F[x]/\langle x - u_0 \rangle \times \cdots \times F[x]/\langle x - u_{r-1} \rangle \cong F^r \\ f & \longmapsto & (f(u_0), \ldots, f(u_{r-1})) \end{array}$$

from Theorem 5.2 is just the evaluation homomorphism (4) at $u_0, \ldots, u_{r-1}$. (The ring F^r consists of the r-tuples with entries from F, and the ring operations are done coordinatewise.) Moreover, the l_i from the proof of Theorem 5.2 satisfying

$$\begin{aligned} l_i &\equiv l_i(u_i) = 1 \bmod (x - u_i), \\ l_i &\equiv l_i(u_j) = 0 \bmod (x - u_j) \text{ for } j \neq i \end{aligned}$$

and $\deg l_i < r$ are the Lagrange interpolants

$$l_i = \prod_{\substack{0 \leq j < r \\ j \neq i}} \frac{x - u_j}{u_i - u_j}.$$

If $v_0, \ldots, v_{r-1} \in F$ are scalars, then $c_i = v_i s_i$ in step 2 of Algorithm 5.4, and

$$f = \sum_{0 \leq i < r} c_i \frac{m}{m_i} = \sum_{0 \leq i < r} v_i l_i$$

is nothing but the familiar Lagrange interpolation polynomial satisfying

$$f(u_i) = v_i \text{ for } 0 \leq i < r. \tag{7}$$

Thus Chinese remaindering for r distinct monic linear polynomials is the same as interpolation at r points, and the CRT tells us once more what we already know: the interpolation polynomial is unique modulo $m = \prod_{0 \leq i < r} (x - u_i)$, so that there is exactly one polynomial $f \in F[x]$ of degree strictly less than r which solves the interpolation problem (7). In fact, it is useful to think of the CRT as a generalization of interpolation. $\diamond$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

With $R = \mathbb{Z}$ and the m_i as in (i) of the above example, we obtain the following formula for Euler's totient function from (2) of Section 4.2 and the Chinese Remainder Theorem. Exercise 5.28 gives the corresponding formula for polynomials over a finite field.

COROLLARY 5.6.
If $m = p_0^{e_0} \cdots p_{r-1}^{e_{r-1}}$, with distinct primes $p_0, \ldots, p_{r-1} \in \mathbb{N}$ and $e_0, \ldots, e_{r-1} \in \mathbb{N}_{>0}$, then

$$\varphi(m) = (p_0 - 1)p_0^{e_0 - 1} \cdots (p_{r-1} - 1)p_{r-1}^{e_{r-1} - 1} = m \cdot \prod_{p \mid m,\ p \text{ prime}} \left(1 - \frac{1}{p}\right).$$

THEOREM 5.7.
Let $R = F[x]$ for a field F, $m_0, \ldots, m_{r-1}, m \in R$ as in (5), $d_i = \deg m_i \geq 1$ for $0 \leq i < r$, $n = \deg m = \sum_{0 \leq i < r} d_i$, and $v_i \in R$ with $\deg v_i < d_i$. Then the unique solution $f \in F[x]$ with $\deg f < n$ of the Chinese Remainder Problem

$$f \equiv v_i \bmod m_i \text{ for } 0 \leq i < r$$

for polynomials can be computed using $O(n^2)$ operations in F.

PROOF. In step 1 of Algorithm 5.4, we first successively compute $m_0, m_0 m_1, \ldots, m_0 \cdots m_{r-1}$ with at most

$$2 \sum_{1 \leq i < r} (d_0 + \ldots + d_{i-1} + 1)(d_i + 1) = 2 \sum_{0 \leq j < i < r} d_j(d_i + 1) + 2 \sum_{1 \leq i < r} (d_i + 1)$$

$$< 2 \sum_{0 \leq i, j < r} d_j(d_i + 1) = 2 \left(\sum_{0 \leq j < r} d_j \right) \left(\sum_{0 \leq i < r} (d_i + 1) \right) = 2n(n + r) \in O(n^2)$$

operations in F (Section 2.3).

Next, we compute m/m_i for $0 \leq i < r$ in step 2. Each division takes at most $(2d_i + 1)(n - d_i + 1)$ operations, as shown in Section 2.4, and together we have

$$\sum_{0 \leq i < r} (2d_i + 1)(n - d_i + 1) < 2n \sum_{0 \leq i < r} (d_i + 1) = 2n(n + r) \in O(n^2)$$

operations.

We fix $i \in \{0, \ldots, r - 1\}$ in step 2. The Extended Euclidean Algorithm with input m/m_i and m_i takes $O(d_i(n - d_i))$ operations (Theorem 3.16). By the degree formula for s_i (Lemma 3.15 (b)), we have $\deg s_i < \deg m_i = d_i$, and hence the multiplication of v_i and s_i, together with the subsequent division with remainder

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

by m_i, takes $O(d_i^2)$ operations. So we have $O(d_i n)$ operations for each i, and $O(n^2)$ for step 2.

Finally, in step 3 we need $O(d_i(n-d_i))$ operations for the multiplication of c_i and m/m_i for $0 \le i < r$, and $O(rn)$ for the addition of all the products (their degree is strictly less than n). This gives a cost of $O(n^2)$ for step 3, and also a total cost of $O(n^2)$ for the whole algorithm. □

The following is the integer analog of Theorem 5.7; see Exercise 5.29.

THEOREM 5.8.
Let $R = \mathbb{Z}$, $m_0, \ldots, m_{r-1}, m \in \mathbb{N}$ as in (5), $n = \lfloor \log_2 m/64 \rfloor + 1$ the word length of m, and $v_i \in \mathbb{Z}$ such that $0 \le v_i < m_i$ for $0 \le i < r$. Then the unique solution $f \in \mathbb{Z}$ with $0 \le f < m$ of the Chinese Remainder Problem

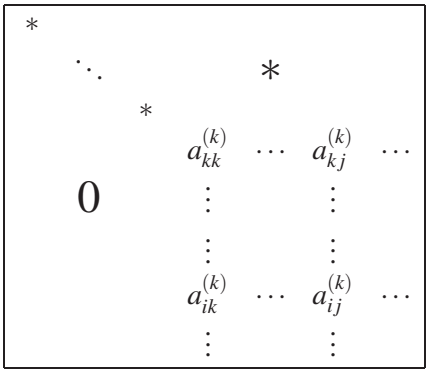
$$f \equiv v_i \bmod m_i \text{ for } 0 \le i < r$$

for integers can be computed using $O(n^2)$ word operations.

5.5. Modular determinant computation

We will use our tools from Section 5.4 on an innocuous problem, namely computing the determinant $\det A \in \mathbb{Z}$ of an $n \times n$ matrix $A = (a_{ij})_{1 \le i,j \le n} \in \mathbb{Z}^{n \times n}$.

We know from linear algebra (Section 25.5) that this problem can be solved by means of Gaussian elimination over $\mathbb{Q}$, which costs at most $2n^3$ operations in $\mathbb{Q}$. Is this "polynomial time"? Of course $2n^3$ is polynomial in the input size, but the number of word operations that the algorithm uses will also depend on the numerators and denominators of the intermediate results. How large can these grow? We consider the kth stage during the elimination, and suppose for simplicity that A is nonsingular and that no row or column permutations are necessary.



The table represents the matrix after $k-1$ pivoting stages, a "$*$" denotes an arbitrary rational number, and the upper diagonal entries are nonzero. The diagonal

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

element $a_{kk}^{(k)} \neq 0$ is the new pivot element, and the entries of the kth column below the pivot element must be made zero in the kth stage by subtracting an appropriate multiple of the kth row. The entries of the matrix for $k < i \leq n$ and $k \leq j \leq n$ change according to the formula

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}} a_{kj}^{(k)}. \tag{8}$$

The entries $a_{ij}^{(1)} = a_{ij}$ are the entries of the original matrix A. If b_k is an upper bound for the absolute value of the numerators and denominators of all $a_{ij}^{(k)}$ for $1 \leq i, j \leq n$, so that in particular $|a_{ij}| \leq b_1$ for $1 \leq i, j \leq n$, then the formula (8) gives

$$b_k \leq 2b_{k-1}^4 \leq 2^{1+4} b_{k-2}^{4^2} \leq \cdots \leq 2^{1+4+\cdots+4^{k-2}} b_1^{4^{k-1}} = 2^{(4^{k-1}-1)/3} b_1^{4^{k-1}},$$

which is an exponentially large upper bound in the input size $n^2 \lambda(b_1) \approx n^2 \log_{2^{64}} b_1$ (see Sections 2.1 and 6.1 concerning the length λ). At this point, we may wonder whether Gaussian elimination indeed uses polynomial time, if we count word operations. In fact, the length of the intermediate results and the number of word operations for Gaussian elimination over $\mathbb{Q}$ are polynomial in the input size, but the proof is nontrivial. We use an alternative approach to reach the same goal, a polynomial time algorithm for computing $\det A$. This illustrates modular computation in a simple case, and introduces some tools of more general interest.

The simplest way of obtaining a polynomial-time computation for the determinant $d = \det A$ of a matrix $A \in \mathbb{Z}^{n \times n}$ is to choose a prime p which is guaranteed to be bigger than $2|d|$, perform Gaussian elimination on $A \bmod p \in \mathbb{Z}_p^{n \times n}$ to calculate $d \bmod p$, and represent this value in the "symmetric" system

$$-\frac{p-1}{2}, \ldots, \frac{p-1}{2} \tag{9}$$

of representatives (if p is odd; see Section 4.1). If $r \in \mathbb{Z}$ is this representative, then

$$r \equiv d \bmod p, \quad -\frac{p}{2} < r < \frac{p}{2}.$$

The congruence holds since any polynomial expression like the determinant commutes with the canonical homomorphism $\mathbb{Z} \longrightarrow \mathbb{Z}_p$ (Section 25.3); so that the determinant, taken modulo p, of A equals the determinant in $\mathbb{Z}_p$ of the matrix $(A \bmod p) \in \mathbb{Z}_p^{n \times n}$ whose entries are those of A, taken modulo p. It follows that p divides $d - r$,

$$|d - r| \leq |d| + |r| < \frac{p}{2} + \frac{p}{2} = p,$$

and hence $d = r$.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

We note the trivial but surprisingly useful fact that we used:

$$\text{if } a \mid b \text{ and } |b| < |a|, \text{ then } b = 0, \text{ for } a, b \in \mathbb{Z}. \tag{10}$$

The calculation of $\det(A \bmod p)$ is essentially the same as Gaussian elimination over $\mathbb{Q}$, except that when dividing by a pivot element a we have to calculate its inverse modulo p by the (traditional) Extended Euclidean Algorithm 3.6. What have we gained? The big win is that we do not have to worry about the growth of the intermediate values anymore: they can always be represented in the system (9), and thus are always "small". It remains to determine a "good" a priori bound on $|\det A|$, that is, one that is only polynomially large in n and the size of the coefficients of A, and which is easy to find without actually calculating $\det A$. Such a bound is provided by **Hadamard's inequality** (Theorem 16.6) which says that $|\det A| \le n^{n/2} B^n$, where $B = \max_{1 \le i,j \le n} |a_{ij}| \in \mathbb{N}$ is the maximal absolute value of an entry of A.

EXAMPLE 5.9. We let $A = \begin{pmatrix} 4 & 5 \\ 6 & -7 \end{pmatrix}$. After Gaussian elimination, the matrix has the form

$$\begin{pmatrix} 4 & 5 \\ 0 & -29/2 \end{pmatrix},$$

so that $\det A = -58$.

The Hadamard bound $|\det A| \le 2^1 7^2 = 98$ is reasonably close to $|\det A| = 58$. So for a modular computation, we choose a prime $p > 2 \cdot 98$, say $p = 199$, and perform Gaussian elimination modulo p. The inverse of the pivot element 4 is 50, and

$$\det(A \bmod 199) = \det \begin{pmatrix} 4 & 5 \\ 0 & 85 \end{pmatrix} = 141 = -58 \text{ in } \mathbb{Z}_{199}. \ \diamond$$

The word length $\lambda(C)$ of the bound $C = n^{n/2} B^n$ on $|\det A|$ is about $\frac{1}{64} \log_2 C = \frac{1}{64} n (\frac{1}{2} \log_2 n + \log_2 B)$, and thus polynomial in the input size $n^2 \lambda(B)$, and we will see in Section 18.4 that a prime p between, say, $2C$ and $4C$ can be found easily with a probabilistic polynomial-time algorithm. Then arithmetic modulo p can be performed in polynomial time, in fact, with $O(\log^2 C)$ word operations. All entries of A are less than p in absolute value, and nothing happens computationally in reducing them modulo p. Thus the cost of the algorithm is $O(n^3)$ operations modulo p, which shows that the determinant of an integer matrix can be computed with

$$O(n^3 \cdot n^2 (\log n + \log B)^2) \text{ or } O^{\sim}(n^5 \log^2 B) \tag{11}$$

word operations, where the $O^{\sim}$ notation ignores logarithmic factors (Section 25.7). This is polynomial time, but not cubic (in n)! Using the fast integer arithmetic of

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

Part II, the running time can be reduced to $O^{\sim}(n^4 \log B)$; this is softly quadratic in the input size. Well, should we call the running time of Gaussian elimination quadratic, cubic, quartic, or quintic?

The algorithm indicated above is not much progress over Gaussian elimination in $\mathbb{Q}$, except that we could easily prove that it works in polynomial time. The really big idea, however, is not to compute with a single modulus, but with several moduli at a time: **small primes modular computation**. These primes can then be chosen very small, of only logarithmic length, and the main cost of the resulting algorithm are many small Gaussian eliminations, which can be performed in a parallel or even distributed fashion. This method is much more efficient.

ALGORITHM 5.10 Small primes modular determinant computation.
Input: $A = (a_{ij})_{1 \le i,j \le n} \in \mathbb{Z}^{n \times n}$, with $|a_{ij}| \le B$ for all i, j.
Output: $\det A \in \mathbb{Z}$.

1. $C \longleftarrow n^{n/2} B^n, \quad r \longleftarrow \lceil \log_2(2C+1) \rceil$
   choose r distinct prime numbers $m_0, \ldots, m_{r-1} \in \mathbb{N}$
2. **for** $i = 0, \ldots, r-1$ compute $A \bmod m_i$
3. **for** $i = 0, \ldots, r-1$ **do**
   compute $d_i \in \{0, \ldots, m_i - 1\}$ such that $d_i \equiv \det A \bmod m_i$ using Gaussian elimination over $\mathbb{Z}_{m_i}$
4. **call** the Chinese Remainder Algorithm 5.4 to determine $d \in \mathbb{Z}$ of least absolute value with $d \equiv d_i \bmod m_i$ for $0 \le i < r$
5. **return** d

Because $\det A$ is a polynomial expression in the coefficients of A, we have $\det A \equiv d_i \bmod m_i$ for $0 \le i < r$ and hence $\det A \equiv d \bmod m$ by the Chinese Remainder Theorem, where $m = m_0 \cdots m_{r-1}$. Since $m \ge 2^r > 2n^{n/2} B^n \ge 2|d|$, we actually have $d = \det A$, as before.

EXAMPLE 5.11. We take the first four prime numbers as moduli and get

$$\begin{aligned} \det A &\equiv 0 \bmod 2, & \det A &\equiv 2 \bmod 3, \\ \det A &\equiv 2 \bmod 5, & \det A &\equiv -2 \bmod 7. \end{aligned}$$

We have $m = 2 \cdot 3 \cdot 5 \cdot 7 = 210$, the solutions to the Chinese Remainder Problem $d \equiv d_i \bmod m_i$ for $1 \le i \le 4$ are $d \in -58 + 210\mathbb{Z} = \{\ldots, -268, -58, 152, 362, \ldots\}$, and the correct solution -58 is the one of least absolute value. If we had taken only the first three primes, we would have incorrectly computed $d = 2$. $\diamond$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

For the cost analysis, Theorem 18.10 says that we can calculate the first r primes using $O(r\log^2 r\log\log r)$ word operations, and that $\log m_i \in O(\log r)$ for all i. (Actually, somewhat fewer than r primes are sufficient, see Exercise 18.21.) Thus $\log m = \sum_{0\le i<r}\log m_i \in O(r\log r)$. A single arithmetic operation modulo m_i can be done with $O(\log^2 m_i)$ or $O(\log^2 r)$ word operations, and hence the total cost of all Gaussian eliminations in step 3 is $O(n^3 r\log^2 r)$ word operations. The reduction of an entry of A modulo one modulus m_i takes $O(\lambda(B)\lambda(m_i))$ or $O(\log B\cdot\log r)$ word operations, by Section 2.4. Therefore the reduction of all entries of A modulo $m_0,\ldots,m_{r-1}$ in step 2 takes $O(n^2\log B\cdot r\log r)$ word operations. The cost for step 4 is $O(r^2\log^2 r)$ word operations, by Theorem 5.8, and dominates the cost of step 1. The fact that $r\in O(n\log(nB))$ leads to the following theorem, which says that the small primes approach is faster by about two orders of magnitude than the big prime algorithm.

THEOREM 5.12.
The determinant of a matrix $A\in\mathbb{Z}^{n\times n}$ with all entries less than B in absolute value can be computed deterministically with

$$O((n^4\log(nB)+n^3\log^2(nB))(\log^2 n+(\log\log B)^2)) \text{ or } O^{\sim}(n^4\log B+n^3\log^2 B)$$

word operations.

In practice, one would precompute and store not the first r primes but r single precision primes close to the word size of the processor (say between 2^{63} and $2^{64}-1$ if the word size is 64). Exercise 18.18 shows that there are sufficiently many such single precision primes for all practical purposes. Then one operation modulo an m_i takes constant time, and the total cost is $O(n^3 r)$, plus $O(n^2\log B\cdot r+r^2)$ or $O(nr^2)$ for the initial modular reduction and the CRA, where r is about $\lambda(2C)$ or $O(n\log(nB))$. In contrast, the cost of the big prime variant is about $O(n^3 r^2)$ word operations.

Similarly to the integer case, a modular algorithm for computing determinants of matrices with entries in $F[x]$, where F is a field, can be designed. If the field is large enough, then this is even easier than the integer case (Exercise 5.32).

5.6. Hermite interpolation

Sections 5.6 through 5.11 are not essential for the rest of the text and may be skipped at first reading. In this section, we discuss an application of the Chinese Remainder Algorithm to **Hermite interpolation**. This is a generalization of polynomial interpolation where at each point not only the value of a function is prescribed, but also the values of some of the first few derivatives, or equivalently, an initial segment of the Taylor expansion.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

If R is an arbitrary (commutative) ring, $f \in R[x]$ has degree at most n, and $u \in R$, then the **Taylor expansion of f around u** is

$$f = f_n \cdot (x-u)^n + \cdots + f_1 \cdot (x-u) + f_0, \tag{12}$$

where $f_n, \ldots, f_0 \in R$ are the **Taylor coefficients**. If $u = 0$, then this is just our usual way of writing polynomials. The Taylor coefficients are uniquely determined, and we have $f_n x^n + \cdots + f_1 x + f_0 = f(x+u)$, which is the polynomial f with x substituted by $x+u$. (Formally, this defines $f_n, \ldots, f_0 \in R[u]$, when we consider u as an indeterminate over R, and then (12) holds for this indeterminate and also for each value from R substituted for it.) For $R = \mathbb{Z}, \mathbb{Q}, \mathbb{R}$, or $\mathbb{C}$, the ith Taylor coefficient of f is equal to $f^{(i)}(u)/i!$, where $f^{(i)}$ is the ith derivative of f with respect to x, and (12) takes the more familiar form

$$f = \frac{f^{(n)}(u)}{n!} \cdot (x-u)^n + \cdots + \frac{f''(u)}{2} \cdot (x-u)^2 + f'(u) \cdot (x-u) + f(u).$$

Thus for $f \in \mathbb{Z}[x]$ and $u \in \mathbb{Z}$, $f^{(i)}(u)/i!$ is always an integer. For $e \leq n$, (12) implies that

$$f \equiv f_{e-1} \cdot (x-u)^{e-1} + \cdots + f_1 \cdot (x-u) + f_0 \bmod (x-u)^e. \tag{13}$$

We say that the right hand side of (13) is the **Taylor expansion of f around u of order e**.

Let F be a field, $u_0, \ldots, u_{r-1} \in F$ distinct, $e_0, \ldots, e_{r-1} \in \mathbb{N}$, and $v_0, \ldots, v_{r-1} \in F[x]$ with $\deg v_i < e_i$ for all i. The **Hermite interpolation problem** is then to compute a polynomial $f \in F[x]$ of degree less than $n = e_0 + \cdots + e_{r-1}$ such that for all i, v_i is the Taylor expansion of f around u_i of order e_i, or equivalently, f solves the Chinese Remainder Problem

$$f \equiv v_i \bmod (x-u_i)^{e_i} \text{ for } 0 \leq i < r. \tag{14}$$

EXAMPLE 5.13. We look for a polynomial $f \in \mathbb{Q}[x]$ of degree less than 4 such that

$$f(0) = 0,\ f'(0) = 1,\ f(1) = 1,\ f'(1) = 0, \tag{15}$$

Thus the initial segments of the Taylor expansions of f at $x = 0$ and $x = 1$ are $v_0 = f(0) + f'(0)x = x$ and $v_1 = f(1) + f'(1)(x-1) = 1$, respectively, and the conditions (15) are equivalent to the congruences

$$f \equiv x \bmod x^2, \quad f \equiv 1 \bmod (x-1)^2.$$

Here our moduli are $m_0 = x^2$ and $m_1 = (x-1)^2$, and the Extended Euclidean Algorithm finds that $(-2x+3)x^2 + (2x+1)(x-1)^2 = 1$. Thus $s_0 = 2x+1$ and $s_1 = -2x+3$ in step 2 of the Chinese Remainder Algorithm 5.4,

$$c_0 = v_0 s_0 \text{ rem } m_0 = x \cdot (2x+1) \text{ rem } x^2 = x,$$
$$c_1 = v_1 s_1 \text{ rem } m_1 = 1 \cdot (-2x+3) \text{ rem } (x-1)^2 = -2x+3,$$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

and finally

$$f = c_0\frac{m}{m_0} + c_1\frac{m}{m_1} = x(x-1)^2 + (-2x+3)x^2 = -x^3 + x^2 + x.$$

Thus $f' = -3x^2 + 2x + 1$, and we easily check that (15) is satisfied. $\diamond$

The Chinese Remainder Theorem 5.7 implies the following.

COROLLARY 5.14.
The Hermite interpolation problem (14) can be solved using $O(n^2)$ arithmetic operations in F.

5.7. Rational function reconstruction

In this section, we solve the problem of finding a rational function of small "degree" that is congruent to some polynomial modulo another polynomial. As a consequence, we obtain solutions for various interpolation problems, and in the most general form a "rational" Chinese Remainder Algorithm.

Let F be a field, $m \in F[x]$ of degree $n > 0$, and $g \in F[x]$ of degree less than n. For a given $k \in \{0,\ldots,n\}$, we want to find a rational function $r/t \in F(x)$, with $r,t \in F[x]$, satisfying

$$\gcd(t,m) = 1 \text{ and } rt^{-1} \equiv g \bmod m, \quad \deg r < k, \quad \deg t \le n-k, \tag{16}$$

where t^{-1} is the inverse of t modulo m (Section 4.2). If $k = n$, then clearly $r = g$ and $t = 1$ is a solution, but it is not clear at all whether solutions exist for other values of k. The degree constraints in (16) are "natural" in the sense that, for fixed m, the input g has n coefficients, and the constraints leave exactly n "degrees of freedom" for the coefficients of r and t.

Since t is a unit modulo m, we may multiply the congruence in (16) by t to obtain an equivalent condition. When we drop the gcd requirement, we obtain

$$r \equiv tg \bmod m, \quad \deg r < k, \quad \deg t \le n-k. \tag{17}$$

Now this is a strictly weaker condition, and we will see that it can always be satisfied, but that there are (exceptional) cases where (16) has no solution.

The following lemma is in a sense the converse of Lemma 3.15 (b), which state that the s_i, t_i in the Extended Euclidean Algorithm have small degrees. It says that any linear combination $r = sf + tg$ of f and g, where $f,g,r,s,t \in F[x]$ and the degrees of r,s,t are "small", is a multiple of some row $r_j = s_j f + t_j g$ in the EEA.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

LEMMA 5.15 *(Uniqueness of the EEA entries). Let F be a field, $f,g,r,s,t \in F[x]$ with $\deg f = n$, $r = sf + tg$ and $t \neq 0$, and suppose that*

$$\deg r + \deg t < n = \deg f.$$

Moreover, let r_i, s_i, t_i for $0 \leq i \leq \ell + 1$ be the rows of the Extended Euclidean Algorithm 3.14 for the pair (f,g). If we define $j \in \{1, \ldots, \ell+1\}$ by

$$\deg r_j \leq \deg r < \deg r_{j-1}, \tag{18}$$

then there exists a nonzero $\alpha \in F[x]$ such that

$$r = \alpha r_j,\ s = \alpha s_j,\ t = \alpha t_j.$$

PROOF. First, we claim that $s_j t = s t_j$. Suppose that the claim is false, and consider the equation

$$\begin{pmatrix} s_j & t_j \\ s & t \end{pmatrix} \begin{pmatrix} f \\ g \end{pmatrix} = \begin{pmatrix} r_j \\ r \end{pmatrix}.$$

The coefficient matrix is nonsingular, and we can solve for f in $F(x)$ using Cramer's rule (Theorem 25.6), obtaining

$$f = \frac{\det \begin{pmatrix} r_j & t_j \\ r & t \end{pmatrix}}{\det \begin{pmatrix} s_j & t_j \\ s & t \end{pmatrix}}. \tag{19}$$

The degree of the left hand side of (19) is n, while

$$\begin{aligned} \deg(r_j t - r t_j) &\leq \max\{\deg r_j + \deg t, \deg r + \deg t_j\} \\ &\leq \max\{\deg r + \deg t, \deg r + n - \deg r_{j-1}\} \\ &< \max\{n, \deg r_{j-1} + n - \deg r_{j-1}\} = n, \end{aligned}$$

by Lemma 3.15 (b) and (18), and the degree of the right hand side of (19) is strictly less than n. Thus we have a contradiction, proving the claim.

Now, Lemma 3.15 (v) implies that s_j and t_j are relatively prime, and from the claim we have $t_j \mid s_j t$, so that $t_j \mid t$. We write $t = \alpha t_j$, where $\alpha \in F[x]$, and $\alpha \neq 0$ since $t \neq 0$. Then we have $s t_j = s_j t = \alpha s_j t_j$, and cancelling t_j, we obtain $s = \alpha s_j$. Finally,

$$r = sf + tg = \alpha(s_j f + t_j g) = \alpha r_j. \ \square$$

We now show that (17) can be solved by means of the Extended Euclidean Algorithm. We say that a rational function $r/t \in F(x)$, with $r,t \in F[x]$, is in **canonical form** if t is monic and $\gcd(r,t) = 1$. Every rational function has a unique canonical form.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

THEOREM 5.16.
Let $m \in F[x]$ of degree $n > 0$ and $g \in F[x]$ of degree less than n. Furthermore, let $r_j, s_j, t_j \in F[x]$ be the jth row in the Extended Euclidean Algorithm for m, g, where j is minimal such that $\deg r_j < k$.

(i) There exist polynomials $r, t \in F[x]$ satisfying (17), namely $r = r_j$ and $t = t_j$. If in addition $\gcd(r_j, t_j) = 1$, then r and t also solve (16).

(ii) If $r/t \in F(x)$ is a canonical form solution to (16), then $r = \tau^{-1} r_j$ and $t = \tau^{-1} t_j$, where $\tau = \mathrm{lc}(t_j) \in F \setminus \{0\}$. In particular, (16) is solvable if and only if $\gcd(r_j, t_j) = 1$.

PROOF. (i) We have $r_j = s_j m + t_j g \equiv t_j g \bmod m$ and

$$\deg t_j = \deg r_0 - \deg r_{j-1} = n - \deg r_{j-1} \leq n - k,$$

by Lemma 3.15 (b) and the minimality of j, and hence $(r, t) = (r_j, t_j)$ satisfy (17). Here, r_{j-1} is the remainder in the EEA preceding r_j. Finally, Lemma 3.15 (vi) implies that $\gcd(m, t_j) = \gcd(r_j, t_j) = 1$, and the claim follows.

(ii) Let $s \in F[x]$ such that $r = sm + tg$. The degree constraints in (16) imply that $\deg r + \deg t < n = \deg m$. From Lemma 5.15, we conclude that $(r, t) = (\alpha r_j, \alpha t_j)$ for some nonzero $\alpha \in F[x]$. Since r and t are relatively prime and t is monic, $\alpha = \tau^{-1} \in F$ is constant and $\gcd(r_j, t_j) = 1$. □

Using Theorem 3.16, we obtain the following.

COROLLARY 5.17.
There is an algorithm which decides whether (16) is solvable, and if so, computes its unique solution using $O(n^2)$ operations in F.

In the next two sections, several examples will illustrate Theorem 5.16. We will combine the Chinese Remainder Algorithm for polynomials with rational function reconstruction to solve various interpolation problems, depicted in Table 5.5. The problems will be precisely specified below. When the input consists of n items, we also have a degree constraint on the output which leaves n choices. Then the polynomial problems always have a solution, and the rational ones typically do, but not always. The polynomial output in the second and the fourth row is trivially just the input. The next-to-last column in Table 5.5 can be considered as a special case of the last column. The third row is the "least common generalization" of the first two rows and a special case of the last row. The solution of these problems proceeds in two steps.

- First a polynomial solution is computed, by the Chinese Remainder Algorithm. This was done in the preceding sections.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

input	moduli	polynomial output	rational function output
several values	$m_i = x - u_i$, u_i distinct	polynomial interpolation, §5.2	Cauchy interpolation, §5.8
Taylor expansion around 0	$m = x^n$		Padé approximation, §5.9
Taylor expansions around several u_i	$m_i = (x-u_i)^{e_i}$, u_i distinct	Hermite interpolation, §5.6	rational Hermite interpolation, Exercises 5.42, 5.43
remainder mod m	m arbitrary		rational function reconstruction, §5.7
remainders modulo several m_i	m_i arbitrary, pairwise coprime	CRA, §5.4	rational CRA, Exercise 5.42

TABLE 5.5: Various interpolation problems.

- Then the required rational solution is calculated, via the Extended Euclidean Algorithm for the polynomial solution and a problem-specific modulus.

Like polynomial interpolation, Cauchy and Hermite interpolation and Padé approximation are well-studied problems in numerical analysis. The various designations illustrate the power of our general approach: people had found each of these problems interesting and studied them, and only in hindsight can we classify them as special instances of the single general task "rational CRA".

5.8. Cauchy interpolation

The polynomial interpolation problem is, given a collection of sample values $v_i = f(u_i) \in F$ for $0 \le i < n$ of an unknown function $f: F \longrightarrow F$ at distinct points $u_0, \ldots, u_{n-1}$ of a field F, to compute a polynomial $g \in F[x]$ of degree less than n that interpolates g at those points, so that $g(u_i) = v_i$ for all i. We saw in Section 5.2 that such a polynomial always exists uniquely and learned how to compute it using the Lagrange interpolation formula.

A more general problem is **Cauchy interpolation** or rational interpolation, where furthermore $k \in \{0, \ldots, n\}$ is given and we are looking for a rational function $r/t \in F(x)$, with $r, t \in F[x]$, such that

$$t(u_i) \neq 0 \text{ and } \frac{r(u_i)}{t(u_i)} = v_i \text{ for } 0 \le i < n, \quad \deg r < k, \quad \deg t \le n - k. \tag{20}$$

Like polynomial interpolation, Cauchy interpolation can be used to approximate real-valued functions given only by their values at a finite set of points. Empirically, it is often the case that the approximation error is smaller for rational functions than for polynomials, in particular, when the function to be approximated has singularities; we will see an example below.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

Obviously $t = 1$ and $r = g$, where g is an interpolating polynomial as above, is a solution to (20) for $k = n$, but it is not clear whether solutions for other values of k exist. Multiplying (20) by $t(u_i)$ and dropping the requirement that it be nonzero, we obtain the weaker condition

$$r(u_i) = t(u_i)v_i \text{ for } 0 \le i < n, \quad \deg r < k, \quad \deg t \le n-k. \tag{21}$$

Now for any i, $r(u_i) = t(u_i)v_i = t(u_i)g(u_i)$ if and only if $r \equiv tg \bmod (x-u_i)$, and by the Chinese Remainder Theorem, Corollary 5.3, (21) is in turn equivalent to (17) with $m = (x-u_0)\cdots(x-u_{n-1})$. The following consequence of Theorem 5.16 on rational function reconstruction gives a complete answer on existence and uniqueness of a solution to (20).

COROLLARY 5.18.
Let F be a field, $u_0,\ldots,u_{n-1} \in F$ be distinct, $v_0,\ldots,v_{n-1} \in F$, $g \in F[x]$ of degree less than n with $g(u_i) = v_i$ for all i, and $k \in \{0,\ldots,n\}$. Furthermore, let $r_j, s_j, t_j \in F[x]$ be the jth row in the Extended Euclidean Algorithm for the polynomials $m = (x-u_0)\cdots(x-u_{n-1})$ and g, where j is minimal such that $\deg r_j < k$.

(i) There exist polynomials $r,t \in F[x]$ satisfying (21), namely $r = r_j$ and $t = t_j$. If in addition $\gcd(r_j,t_j) = 1$, then r and t also solve (20).

(ii) If $r/t \in F(x)$ is a canonical form solution to (20), then $r = \tau^{-1}r_j$ and $t = \tau^{-1}t_j$, where $\tau = \mathrm{lc}(t_j) \in F \setminus \{0\}$. In particular, (20) is solvable if and only if $\gcd(r_j,t_j) = 1$.

Thus to find a rational interpolating function as in (20), we compute the entries r_j and t_j in the jth row of the Extended Euclidean Algorithm for m, g, in the notation of the above corollary. If $\gcd(r_j,t_j) = 1$, then r_j/t_j is the unique canonical form solution to the rational interpolation problem (20). If the gcd is nontrivial, however, then also $\gcd(m,t_j) \ne 1$, by Lemma 3.15 (vi). Thus $t_j(u_i) = 0$ for some $i \in \{0,\ldots,n-1\}$, and (20) has no solution. (In numerical analysis, such a u_i is called an unreachable point.) A trivial example for the latter is when $k = 0$ and not all v_i are zero, so that $r = 0$ and $t = m$ satisfy (21) but not (20). More interesting is the following.

EXAMPLE 5.19. (i) Let $F = \mathbb{F}_5$, and suppose that we want to compute a rational function $\rho = r/t \in \mathbb{F}_5(x)$, with $r,t \in \mathbb{F}_5[x]$ of degree at most one, such that $\rho(i) = 2^i$ for $i = 0,1,2$. Exercise 5.4 computes the interpolating polynomial $g = 3x^2+3x+1$ of degree less than 3. The Extended Euclidean Algorithm for $m = x(x-1)(x-2) = x^3+2x^2+2x$ and g computes

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

j	q_j	ρ_j	r_j	s_j	t_j
0		1	x^3+2x^2+2x	1	0
1	$x+1$	3	x^2+x+2	0	2
2	$x+4$	4	$x+2$	4	$2x+2$
3	$x+2$	4	1	$4x+1$	$2x^2+1$
4		1	0	x^2+x+2	$3x^3+x^2+x$

and from row 2 we get the desired rational function

$$\rho = \frac{r_2}{t_2} = \frac{x+2}{2x+2} = \frac{3x+1}{x+1} \in \mathbb{F}_5(x).$$

Row 3 gives another rational interpolating function, namely

$$\rho = \frac{r_3}{t_3} = \frac{1}{2x^2+1} = \frac{3}{x^2+3}.$$

Row 4 would yield $\rho = r_4/t_4 = 0$, but this is obviously not an interpolating function. We have $\gcd(r_4,t_4) = \gcd(m,t_4) = m$.

(ii) Let $F = \mathbb{Q}$, $n = 3$, $u_0 = 0$, $u_1 = 1$, $u_2 = -1$, $v_0 = 1$, $v_1 = 2$, $v_2 = 2$, and suppose that we are looking for a rational function $\rho = r/t \in \mathbb{Q}(x)$ satisfying (20) for $k = 2$. Making the ansatz $r = a_1x + a_0$ and $t = b_1x + b_0$ and plugging in u_0, u_1, u_2, we arrive at the linear system

$$a_0 = b_0, \quad a_1 + a_0 = 2(b_1 + b_0), \quad -a_1 + a_0 = 2(-b_1 + b_0),$$

which is equivalent to (21). It simplifies to

$$a_0 = b_0, \quad 2a_0 = 4b_0, \quad 2a_1 = 4b_1,$$

and hence $r = 2x$, $t = x$ form—up to multiplication by a constant—the unique solution of (21). However, the rational function

$$\rho = \frac{r}{t} = \frac{2x}{x} = 2$$

does not solve (20) since obviously $\rho(u_0) = \rho(0) \neq 1 = v_0$, and hence (20) has no solution.

The Extended Euclidean Algorithm for $m = x(x-1)(x+1) = x^3 - x$ and the interpolating polynomial $x^2 + 1$ yields

j	q_j	ρ_j	r_j	s_j	t_j
0		1	x^3-x	1	0
1	x	1	x^2+1	0	1
2	x	-2	x	$-\frac{1}{2}$	$\frac{1}{2}x$
3	x	1	1	$\frac{1}{2}x$	$-\frac{1}{2}x^2+1$
4		1	0	$-\frac{1}{2}x^2-\frac{1}{2}$	$\frac{1}{2}x^3-\frac{1}{2}x$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

We see from row 2 that $r = x$ and $t = x/2$ solves (21), but we are not allowed to cancel the common factor x since $\rho = 2$ does not solve (20). $\diamond$

The alternative way of solving (21) via a system of linear equations, as we did above, works in general but is less efficient than the EEA.

COROLLARY 5.20.
There is an algorithm that either computes the canonical form solution to (20) or else certifies that (20) is unsolvable, using $O(n^2)$ arithmetic operations in F.

5.9. Padé approximation

Let F be a field and $g = \sum_{i\geq 0} g_i x^i \in F[[x]]$ with all $g_i \in F$ be a formal power series (Section 25.3). A **Padé approximant** to g is a rational function $\rho = r/t \in F(x)$, with $r,t \in F[x]$ and $x \nmid t$, that "approximates" g to a sufficiently high power of x. More precisely, r/t is a **$(k, n-k)$-Padé approximant** to g if

$$x \nmid t \text{ and } \frac{r}{t} \equiv g \bmod x^n, \quad \deg r < k, \quad \deg t \leq n-k; \tag{22}$$

the congruence is equivalent to $r \equiv tg \bmod x^n$. Obviously $r = \sum_{0\leq i<n} g_i x^i$, the Taylor expansion of order n of g around 0, and $t = 1$ is an $(n,0)$-Padé approximant for each $n \in \mathbb{N}$, but it is not clear whether approximants for $k < n$ exist. A more general question is to ask for Padé approximants around u of a formal power series in $x-u$ for an arbitrary $u \in F$. This may be reduced to (22) by performing the shift of variable $x \longmapsto x+u$.

In numerical analysis, one is interested in approximating arbitrary (sufficiently smooth) real-valued functions by "simple" functions such as polynomials or rational functions. Taylor expansions and Padé approximants provide such approximations in the vicinity of the origin (or any other point, after an appropriate change of variable). As in the case of interpolation, it was observed empirically that sometimes rational functions yield a much smaller approximation error, in particular when the function to be approximated has singularities; see Example 5.23 below.

The similarity with Cauchy interpolation is clear: instead of prescribing the values of ρ at n distinct points $u_0, \ldots, u_{n-1}$, we have $u_0 = \cdots = u_{n-1} = 0$ and prescribe an initial segment of the Taylor expansion of ρ at u_0. Indeed the statements of the previous section carry over almost literally if we replace $m = (x-u_0)\cdots(x-u_{n-1})$ by $m = x^n$. The following is a consequence of Theorem 5.16.

COROLLARY 5.21.
Let $g \in F[x]$ have degree less than $n \in \mathbb{N}$, $k \in \{0, \ldots, n\}$, and $r_j, s_j, t_j \in F[x]$ be the jth row in the Extended Euclidean Algorithm for $m = x^n$ and g, where j is minimal such that $\deg r_j < k$.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(i) There exist polynomials $r,t \in F[x]$ satisfying

$$r \equiv tg \bmod x^n, \quad \deg r < k, \quad \deg t \leq n-k, \tag{23}$$

namely $r = r_j$ and $t = t_j$. If in addition $\gcd(r_j,t_j) = 1$, then r and t also solve (22).

(ii) If $r/t \in F(x)$ is a canonical form solution to (22), then $r = \tau^{-1}r_j$ and $t = \tau^{-1}t_j$, where $\tau = \mathrm{lc}(t_j) \in F \setminus \{0\}$. In particular, (22) is solvable if and only if $\gcd(r_j,t_j) = 1$.

EXAMPLE 5.22. (i) Let $g = \sum_{i\geq 0}(i+1)x^i = 1+2x+3x^2+4x^3+\cdots \in \mathbb{F}_5[[x]]$, and suppose we want to compute a $(2,2)$-Padé approximant to g. The Extended Euclidean Algorithm 3.14 for $m = x^4$ and $4x^3+3x^2+2x+1 \in \mathbb{F}_5[x]$ yields

j	q_j	ρ_j	r_j	s_j	t_j
0		1	x^4	1	0
1	$x+3$	4	x^3+2x^2+3x+4	0	4
2	x	1	x^2+2x+3	1	$x+3$
3	x^2+2x+3	4	1	x	x^2+3x+1
4		1	0	$4x^3+3x^2+2x+1$	$4x^4$

and we can see in row 3 that

$$\frac{r_3}{t_3} = \frac{1}{x^2+3x+1} = \frac{1}{(x-1)^2}$$

is the required solution. In fact, this is a $(k,n-k)$-Padé approximant to g for all values of $k \geq 1$ and n such that $n-k \geq 2$, since g is the formal derivative of the geometric series $1/(1-x) = \sum_{i\geq 0}x^i$, and hence $g = 1/(x-1)^2$ is the formal power series inverse of $(x-1)^2$.

The above table contains other Padé approximants to g: row 1 gives the trivial $(4,0)$ approximant $r_1/t_1 = 4x^3+3x^2+2x+1$ and row 2 yields the $(3,1)$ approximant $r_2/t_2 = (x^2+2x+3)/(x+3)$, but row 4 does not give a Padé approximant since x divides t_4, and in fact $r_4/t_4 = 0$ does not approximate g.

(ii) Let $g = x^2+1 \in \mathbb{Q}[[x]]$ and $n = 3$. Then there is no $(2,1)$-Padé approximant to g. To see why, we assume that there are polynomials $r,t \in \mathbb{Q}[x]$ of degree at most 1 such that $x \nmid t$ and $r \equiv tg \bmod x^3$. Let $t = ax+b$, with $a,b \in F$ and $b \neq 0$. Then

$$r \equiv (ax+b)(x^2+1) \equiv bx^2+ax+b \bmod x^3,$$

which is impossible since $\deg r \leq 1$.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

The Extended Euclidean Algorithm for $m = x^3$ and x^2+1 computes

j	q_j	ρ_j	r_j	s_j	t_j
0		1	x^3	1	0
1	x	1	x^2+1	0	1
2	x	-1	x	-1	x
3	x	1	1	x	$-x^2+1$
4		1	0	$-x^2-1$	x^3

and from row 2, we see that $r = t = x$ solve (23), but we are not allowed to divide the common factor x out of r and t, since $r/t = 1 \not\equiv x^2+1 \bmod x^3$. $\diamond$

EXAMPLE 5.23. We want to approximate the tangent function in the vicinity of the origin with an approximation error of $O(x^9)$ when $x \in \mathbb{R}$ approaches 0. The Taylor series of $\tan x$ around the origin is

$$\tan x = x + \frac{1}{3}x^3 + \frac{2}{15}x^5 + \frac{17}{317}x^7 + \cdots$$

and converges for all $x \in \mathbb{C}$ such that $|x| < \pi/2$. At $x = \pm\pi/2 \approx \pm 1.57$, the tangent function has simple poles. The Taylor polynomial of order 9 is

$$g = \frac{17}{317}x^7 + \frac{2}{15}x^5 + \frac{1}{3}x^3 + x \in \mathbb{Q}[x],$$

and using rational function reconstruction, we obtain the Padé approximants of order $n = 9$ in Table 5.6. (We have not made the denominators monic.) The

$k = 9$ or 8	$k = 7$ or 6	$k = 5$ or 4	$k = 3$ or 2
g	$\dfrac{x^5+45x^3-630x}{255x^2-630}$	$\dfrac{-10x^3+105x}{x^4-45x^2+105}$	$\dfrac{-945x}{2x^6+21x^4+315x^2-945}$

TABLE 5.6: The Padé approximants of order 9 to the tangent function.

approximations are so good that in a plot it is hard to tell them apart from the tangent function. Instead, Figure 5.7 on page 124 shows the difference between the tangent function and each of the four Padé approximants from Table 5.6 on the interval $(-\pi/2, \pi/2)$. It can be seen that the Padé approximant for $k = 5$ or 4 is the best one. For example, its approximation error to $\tan(1.5) \approx 14.1$ is about 0.059, while the Taylor polynomial has an approximation error of about 9.54 at that point. $\diamond$

Corollary 5.21 yields a decision procedure for Padé approximants: Compute the appropriate results r_j and t_j of the Extended Euclidean Algorithm. If their gcd is one, then r_j/t_j is the unique $(k, n-k)$-Padé approximant as in (22), otherwise no such approximant exists.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

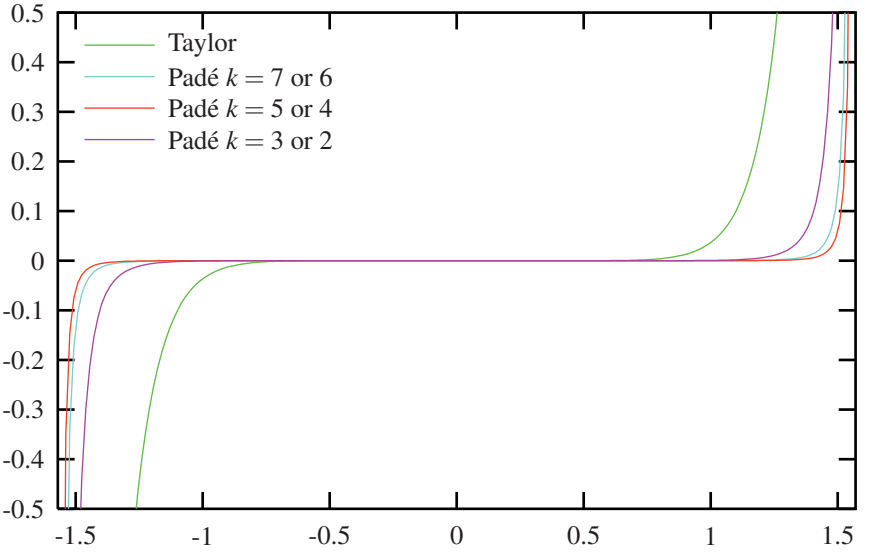


FIGURE 5.7: The difference of $\tan x$ to its Padé approximants of order 9 around the origin.

COROLLARY 5.24.
There is an algorithm that either computes the canonical form solution to (22) or else certifies that (22) is unsolvable, using $O(n^2)$ arithmetic operations in F.

This algorithm will be put to use in Chapter 7. Using the fast Euclidean Algorithm from Chapter 11, the running time drops to $O(n \log^2 n \operatorname{loglog} n)$ arithmetic operations in F.

5.10. Rational number reconstruction

The integer analog of rational function reconstruction is, given integers $m > g \geq 0$ and $k \in \{1, \ldots, m\}$, to compute a rational number $r/t \in \mathbb{Q}$, with $r, t \in \mathbb{Z}$, such that

$$\gcd(t, m) = 1 \text{ and } rt^{-1} \equiv g \bmod m, \quad |r| < k, \quad 0 \leq t \leq \frac{m}{k}, \tag{24}$$

where t^{-1} is the inverse of t modulo m. As in the polynomial case, we will see that the related problem

$$r \equiv tg \bmod m, \quad |r| < k, \quad 0 \leq t \leq \frac{m}{k}, \tag{25}$$

is always solvable, while (24) need not have a solution. The uniqueness statements are a bit weaker than in the polynomial case, however. The following lemma is the integer analog of the Uniqueness Lemma 5.15.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

LEMMA 5.25. *Let $f,g \in \mathbb{N}$ and $r,s,t \in \mathbb{Z}$ with $r = sf + tg$, and suppose that*

$$|r| < k \text{ and } 0 < t \leq \frac{f}{k} \text{ for some } k \in \{1,\ldots,f\}.$$

We let $r_i,s_i,t_i \in \mathbb{Z}$ for $0 \leq i \leq \ell+1$ be the results of the traditional Extended Euclidean Algorithm for f,g, with $r_i \geq 0$ for all i. Moreover, we define $j \in \{1,\ldots,\ell+1\}$ by

$$r_j < k \leq r_{j-1}, \tag{26}$$

and if $j \leq \ell$, we choose $q \in \mathbb{N}_{\geq 1}$ such that

$$r_{j-1} - qr_j < k \leq r_{j-1} - (q-1)r_j,$$

and let $q = 0$ if $j = \ell+1$. Then there exists a nonzero $\alpha \in \mathbb{Z}$ such that

$$\text{either } (r,s,t) = (\alpha r_j, \alpha s_j, \alpha t_j) \text{ or } (r,s,t) = (\alpha r_j^*, \alpha s_j^*, \alpha t_j^*),$$

where $r_j^* = r_{j-1} - qr_j$, $s_j^* = s_{j-1} - qs_j$, and $t_j^* = t_{j-1} - qt_j$.

PROOF. Multiplying $r_j = s_j f + t_j g$ by t and $r = sf + tg$ by t_j and subtracting, we obtain

$$r_j t - r t_j = s_j t f - s t_j f \equiv 0 \bmod f. \tag{27}$$

By (26), we have $0 \leq r_j t < kt \leq f$ and $|rt_j| < kf/r_{j-1} \leq f$, using Lemma 3.12. Thus either $r_j t = rt_j$ or $r_j t = rt_j + f$. In the first case, (27) implies that $s_j t = s t_j$, and since $\gcd(s_j,t_j) = 1$ by Lemma 3.8 (v), we have $t = \alpha t_j$ for some $\alpha \in \mathbb{Z}$; furthermore α is nonzero since t is. Finally, $st_j = s_j t = s_j \alpha t_j$ and $rt_j = r_j t = r_j \alpha t_j$, and hence $s = \alpha s_j$ and $r = \alpha r_j$ since $t_j \neq 0$.

Before we tackle the second case, we note that

$$r_j t_j^* - r_j^* t_j = r_j(t_{j-1} - qt_j) - (r_{j-1} - qr_j)t_j = r_j t_{j-1} - r_{j-1} t_j = (-1)^j f, \tag{28}$$

$$s_j t_j^* - s_j^* t_j = s_j(t_{j-1} - qt_j) - (s_{j-1} - qs_j)t_j = s_j t_{j-1} - s_{j-1} t_j = (-1)^j, \tag{29}$$

by Lemma 3.8. So we assume now that

$$r_j t = rt_j + f. \tag{30}$$

Then $0 \leq r_j t < kt \leq f$ implies $rt_j < 0$, and since $r_\ell = \gcd(f,g)$ divides r, we have $|r| \geq r_\ell$, $j \neq \ell+1$, and $r_j \neq 0$. We claim that $r_j^* t = rt_j^*$. In analogy to (27), we have

$$r_j^* t - r t_j^* = s_j^* t f - s t_j^* f \equiv 0 \bmod f.$$

Since t_{j-1} and t_j alternate in sign (Exercise 3.15), so do t_j and t_j^*, whence $rt_j^* > 0$ and

$$r_j^* t - r t_j^* < kt - r t_j^* \leq f - r t_j^* < f.$$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

On the other hand, using (30) and (28) we obtain

$$r_j(r_j^*t - rt_j^*) = r_j^*(f - rt_j) - r((-1)^j f - r_j^* t_j) = (r_j^* - (-1)^j r)f > -r_j f,$$

since $r_j^* + r_j = r_{j-1} - (q-1)r_j \geq k > |r| \geq (-1)^j r$, by the choice of q. After dividing by the positive integer r_j, we obtain $r_j^* t - rt_j^* > -f$, and the claim follows.

As in the first case, we conclude that $s_j^* t = st_j^*$. Then equation (29) implies that $\gcd(s_j^*, t_j^*) = 1$ and $t = \alpha t_j^*$ for some nonzero $\alpha \in \mathbb{Z}$, and finally $s = \alpha s_j^*$ and $r = \alpha r_j^*$, as above. □

The next theorem is an integer variant of Theorem 5.16. We say that a rational number $r/t \in \mathbb{Q}$, with $r, t \in \mathbb{Z}$, is in **canonical form** if $t > 0$ and $\gcd(r,t) = 1$.

THEOREM 5.26.
Let $g, m \in \mathbb{N}$ with $g < m$, $k \in \{1, \ldots, m\}$, and $r_j, s_j, t_j \in \mathbb{Z}$ be the jth row in the Extended Euclidean Algorithm for m and g, where j is minimal such that $r_j < k$.

(i) There exist $r, t \in \mathbb{Z}$ satisfying (25), namely $(r,t) = (r_j, t_j)$ if $t_j > 0$, and $(r,t) = (-r_j, -t_j)$ otherwise. If in addition $\gcd(r,t) = \gcd(r_j, t_j) = 1$, then r and t also solve (24).

(ii) If $r/t \in \mathbb{Q}$ is a canonical form solution to (24), then either $(r,t) = (\tau r_j, \tau t_j)$ or $(r,t) = (\tau r_j^*, \tau t_j^*)$, where r_j^*, t_j^* are as in Lemma 5.25 and $\tau = \operatorname{sign}(t_j)$ or $\tau = \operatorname{sign}(t_j^*)$, respectively.

(iii) Equation (24) is solvable if and only if $(\gcd(r_j^*, t_j^*) = 1$ and $|t_j^*| \leq m/k)$ or $\gcd(r_j, t_j) = 1$.

(iv) There is at most one canonical form solution to (24) satisfying $|r| < k/2$.

PROOF. (i) We have $r_j = s_j m + t_j g \equiv t_j g \bmod m$, and conclude from Lemma 3.12 that

$$|t_j| \leq \frac{m}{r_{j-1}} \leq \frac{m}{k},$$

by the minimality of j. This proves that $(r,t) = (\pm r_j, \pm t_j)$ solve (25). The proof of the other claim is as in Theorem 5.16.

(ii) Letting $s = (r - tg)/m$, Lemma 5.25 implies that either $(r,t) = (\alpha r_j, \alpha t_j)$ or $(r,t) = (\alpha r_j^*, \alpha t_j^*)$ for some nonzero $\alpha \in \mathbb{Z}$. Furthermore, $\gcd(r,t) = 1$ implies that $\alpha = \pm 1$, and the claim follows.

(iii) By (i), $(\pm r_j, \pm t_j)$ is a canonical form solution of (24) if and only if r_j and t_j are coprime. Since $r_j^* < k$, the pair $(\pm r_j^*, \pm t_j^*)$ is a canonical form solution if and only if $|t_j^*| \leq m/k$ and $\gcd(r_j^*, t_j^*) = 1$. By (ii), these are the only possible solutions.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(iv) Let both r/t and r^*/t^* be canonical form solutions of (24) with $|r| < k/2$ and $|r^*| < k/2$. Since m divides $r - tg$ and $r^* - t^*g$, it also divides $t^*(r - tg) - t(r^* - t^*g) = rt^* - r^*t$. However, $|r|t^* < m/2$ and $|r^*|t < m/2$, whence $rt^* = r^*t$. The claim now follows from $\gcd(r,t) = \gcd(r^*,t^*) = 1$. □

We note that $t_j t_j^* < 0$ (Exercise 3.15) and $r_j, r_j^* \geq 0$, and hence the two possible solutions r_j/t_j and r_j^*/t_j^* of (24) have opposite signs (as rational numbers).

EXAMPLE 5.27. (i) The Extended Euclidean Algorithm for $m = 29$ and $g = 12$ works as follows:

j	q_j	r_j	s_j	t_j
0		29	1	0
1	2	12	0	1
2	2	5	1	−2
3	2	2	−2	5
4	2	1	5	−12
5		0	−12	29

For $k = 10$, the smallest j such that $r_j \leq k$ is $j = 2$, and in fact $r_2/t_2 = -5/2 \equiv 12 \bmod 29$. Now $r_1 - (q-1)r_2 \geq k > r_1 - qr_2$ is satisfied for $q = 1$, whence $r_2^* = r_1 - r_2 = 7$ and $t_2^* = t_1 - t_2 = 3$. But $|t_2^*| = 3 > 29/10 = m/k$, and hence $-5/2$ is the only solution of (25) and (24).

For $k = 9$, j and q are as before, but now $|t_2^*| = 3 \leq 29/9 = m/k$, and we have a second solution $r_2^*/t_2^* = 7/3 \equiv 12 \bmod 29$ of (25) and (24).

(ii) For $m = 22$ and $g = 9$, the Extended Euclidean Algorithm gives

j	q_j	r_j	s_j	t_j
0		22	1	0
1	2	9	0	1
2	2	4	1	−2
3	4	1	−2	5
4		0	9	−22

For $k = 10$, we have $j = 1$, and $r_1/t_1 = 9/1$ is obviously a solution of (24). Now $q = 2$, $(r_1^*, t_2^*) = (r_2, t_2) = (4, -2)$, and $|t_1^*| = 2 \leq 22/10 = m/k$, whence (r_1^*, t_1^*) is a second solution of (25). But $\gcd(r_1^*, t_1^*) = 2$, and $r_1^*/t_1^* = -2$ is not a solution of (24). Thus we have two solutions of (25), but only one of them also solves (24).

For $k = 9$, we have $j = 2$, and $(r_2, t_2) = (4, -2)$ is a solution of (25), but not of (24). Here, $q = 1$ and $(r_2^*, t_2^*) = (r_1 - r_2, t_1 - t_2) = (5, 3)$, but $|t_2^*| = 3 > 22/9 = m/k$, and hence (r_2^*, t_2^*) does not solve (25), so that (25) has a unique solution and (24) is unsolvable.

If $k = 7$, however, then j and q are as before, but now $|t_2^*| = 3 \leq 22/7 = m/k$, and $r_2^*/t_2^* = 5/3$ is the only solution of (24).

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(iii) Let $m = 36$ and $g = 13$. The Extended Euclidean Algorithm for m and g yields

j	q_j	r_j	s_j	t_j
0		36	1	0
1	2	13	0	1
2	1	10	1	−2
3	3	3	−1	3
4	3	1	4	−11
5		0	−13	36

and for $k = 11$ we find that $j = 2$, $(r_2, t_2) = (10, -2)$ solves (25) but not (24), $q = 1$, and $(r_2^*, t_2^*) = (r_1 - r_2, t_1 - t_2) = (3, 3)$ also solves (25) but not (24). Thus (25) has two solutions while (24) has none. ◇

Together with the Chinese Remainder Algorithm for integers, Theorem 5.26 leads to a Chinese Remainder Algorithm for rational numbers (Exercise 5.44).

5.11. Partial fraction decomposition

We discuss another one of the numerous applications of the Chinese Remainder Theorem for polynomials. It will be put to use in Chapter 22.

Let F be a field, $f_1, \ldots, f_r \in F[x]$ nonconstant monic and pairwise coprime polynomials, $e_1, \ldots, e_r \in \mathbb{N}$ positive integers, and $f = f_1^{e_1} \cdots f_r^{e_r}$. (We will see in Part III how to factor polynomials over finite fields and over $\mathbb{Q}$ into irreducible factors, but here we do not assume irreducibility of the f_i.) For another polynomial $g \in F[x]$ of degree less than $n = \deg f$, the **partial fraction decomposition** of the rational function $g/f \in F(x)$ with respect to the given factorization of the denominator f is

$$\frac{g}{f} = \frac{g_{1,1}}{f_1} + \cdots + \frac{g_{1,e_1}}{f_1^{e_1}} + \quad \cdots \quad + \frac{g_{r,1}}{f_r} + \cdots + \frac{g_{r,e_r}}{f_r^{e_r}}, \tag{31}$$

with $g_{ij} \in F[x]$ of smaller degree than f_i, for all i, j. If all f_i are linear polynomials, then the g_{ij} are just constants.

EXAMPLE 5.28. Let $F = \mathbb{Q}$, $f = x^4 - x^2$, and $g = x^3 + 4x^2 - x - 2$. The partial fraction decomposition of g/f with respect to the factorization $f = x^2(x-1)(x+1)$ of f into linear polynomials is

$$\frac{x^3 + 4x^2 - x - 2}{x^4 - x^2} = \frac{1}{x} + \frac{2}{x^2} + \frac{1}{x-1} + \frac{-1}{x+1}. \diamond \tag{32}$$

The following questions pose themselves: Does a decomposition as in (31) always exist uniquely, and how can we compute it? The next lemma is a first step towards an answer.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

LEMMA 5.29. *There exist unique polynomials $c_i \in F[x]$ with $\deg c_i < e_i \deg f_i$ for all i such that*

$$\frac{g}{f} = \frac{c_1}{f_1^{e_1}} + \cdots + \frac{c_r}{f_r^{e_r}}. \tag{33}$$

PROOF. We multiply both sides in (33) by f and obtain the linear equation

$$g = c_1 \prod_{j \neq 1} f_j^{e_j} + \cdots + c_r \prod_{j \neq r} f_j^{e_j} \tag{34}$$

with "unknowns" $c_1, \ldots, c_r$. (We have already seen in Section 4.5 how to find polynomial solutions of such equations.) For any $i \leq r$, each summand with the possible exception of the ith one is divisible by $f_i^{e_i}$, whence $g \equiv c_i \prod_{j \neq i} f_j^{e_j} \bmod f_i^{e_i}$. Now each f_j is coprime to f_i and hence invertible modulo $f_i^{e_i}$, and we obtain

$$c_i \equiv g \prod_{j \neq i} f_j^{-e_j} \bmod f_i^{e_i}, \tag{35}$$

which together with $\deg c_i < \deg f_i^{e_i}$ uniquely determines c_i.

On the other hand, if we define $c_i \in F[x]$ of degree less than $e_i \deg f_i$ by (35), for all i, and let g^* be the right hand side in (34), then g^* is a polynomial of degree less than n and $g^* \equiv g \bmod f_i^{e_i}$ for all i. Now the Chinese Remainder Theorem implies that $g^* \equiv g \bmod f$, and since both polynomials have degree less than n, they are equal. □

It remains to say how to obtain the decomposition (31) from (33). This uses the following generalization of the Taylor expansion. Let R be a ring (commutative, with 1) and $a, p \in R[x]$ with p monic of degree $m > 0$ and a of degree less than km, for some $k, m \in \mathbb{N}$. The ***p*-adic expansion** of a is

$$a = a_{k-1} p^{k-1} + \cdots + a_1 p + a_0, \tag{36}$$

where $a_0, \ldots, a_{k-1} \in R[x]$ have degree less than m. If $p = x - u$ is a linear polynomial, then the a_j are constants, and (36) is just the Taylor expansion of a around u. If a, p are positive integers and $0 \leq a_i < p$ for all i, then (36) is the familiar radix p representation of a.

LEMMA 5.30. *The p-adic expansion exists uniquely, and it can be computed using at most $(km)^2 - km^2$ operations in R.*

PROOF. Dividing a by p with remainder yields $a = qp + r$, with $q, r \in R[x]$ such that $\deg r < m$. Taking $a_0 = r$ and recursively computing the p-adic expansion $q = a_{k-1} p^{k-2} + \cdots + a_1$ of q, we obtain the p-adic expansion of a, proving existence.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

For the uniqueness, let $a = a^*_{k-1}p^{k-1} + \cdots + a^*_1 p + a^*_0$ be another p-adic expansion. Then a^*_0 is the remainder and $a^*_{k-1}p^{k-2} + \cdots + a^*_1$ is the quotient of a on division by p. By induction, the p-adic expansion of the quotient is unique, and hence so is the p-adic expansion of a.

The cost for the first division with remainder is $2\deg p(1 + \deg a - \deg p) \leq 2m^2(k-1)$, and hence the total number of operations in R is at most

$$2m^2 \sum_{1 \leq i \leq k-1} i = m^2(k^2 - k). \; \square$$

Putting it all together, we have the following theorem.

THEOREM 5.31.
The partial fraction decomposition (31) exists uniquely, and it can be computed using $O(n^2)$ operations in F.

PROOF. The existence follows from the preceding two lemmas by taking the f_i-adic expansion $c_i = g_{i,e_i} f_i^{e_i - 1} + \cdots + g_{i,2} f_i + g_{i,1}$ with $g_{ij} \in F[x]$ of degree less than $\deg f_i$ for all i, j. If

$$\frac{g}{f} = \frac{g^*_{1,1}}{f_1} + \cdots + \frac{g^*_{1,e_1}}{f_1^{e_1}} + \cdots + \frac{g^*_{r,1}}{f_r} + \cdots + \frac{g^*_{r,e_r}}{f_r^{e_r}}$$

is another partial fraction decomposition of g/f, with $g^*_{ij} \in F[x]$ of degree less than $\deg f_i$ for all i, j, then Lemma 5.29 implies that $c_i = g^*_{i,e_i} f_i^{e_i - 1} + \cdots + g^*_{i,2} f_i + g^*_{i,1}$ for all i, and the uniqueness of the f_i-adic expansion implies that $g_{ij} = g^*_{ij}$, for all i, j.

To prove the running time bound, we let $d_i = e_i \deg f_i$ and compute $m_i = f_i^{e_i}$ and $v_i = g \text{ rem } f_i^{e_i}$, at a cost of $O(nd_i)$ operations in F altogether, for $1 \leq i \leq r$, taking in total $O(n^2)$ operations. Then we perform steps 1 and 2 of the Chinese Remainder Algorithm 5.4 with input $m_1, \ldots, m_r$ and $v_1, \ldots, v_r$ to compute $c_i \equiv v_i(f/m_i)^{-1} \equiv g(f/f_i^{e_i})^{-1} \bmod m_i$ for all i, taking another $O(n^2)$ operations, by Theorem 5.7. Finally, we compute the f_i-adic expansion of c_i, taking $O(d_i^2)$ operations, by Lemma 5.30, for each i. This is dominated by the cost for the first step, and the claim follows. $\square$

EXAMPLE 5.28 (continued). With f and g as in Example 5.28, we have $m_1 = x^2$, $m_2 = x - 1$, $m_3 = x + 1$, and $f = m_1 m_2 m_3$. Now $v_1 = g \text{ rem } x^2 = -x - 2$, $v_2 = g \text{ rem } x - 1 = 2$, and $v_3 = g \text{ rem } x + 1 = 2$. Then step 2 of the Chinese Remainder Algorithm 5.4 computes

$$c_1 \equiv v_1 \left(\frac{m}{m_1}\right)^{-1} = (-x-2)(x^2-1)^{-1} \equiv (-x-2)(-1)^{-1} = x + 2 \bmod x^2,$$

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

$$c_2 \equiv v_2 \left(\frac{m}{m_2}\right)^{-1} = 2(x^3+x^2)^{-1} \equiv 2\cdot 2^{-1} = 1 \bmod x-1,$$
$$c_3 \equiv v_3 \left(\frac{m}{m_3}\right)^{-1} = 2(x^3-x^2)^{-1} \equiv 2\cdot(-2)^{-1} = -1 \bmod x+1,$$

and hence

$$\frac{x^3+4x^2-x-2}{x^4+x} = \frac{x+2}{x^2} + \frac{1}{x-1} + \frac{-1}{x+1}.$$

Using the x-adic expansion $1\cdot x+2$ of $x+2$ immediately leads to (32). $\diamond$

A different way to compute the partial fraction decomposition (31) is to plug in the g_{ij} with unknown coefficients, clear up denominators, and solve the linear system that arises from comparing coefficients on both sides. The coefficient matrix of the linear system is an $n \times n$ matrix, and its solution using Gaussian elimination takes $O(n^3)$ operations in F (Section 25.5). However, the method suggested in the above theorem is faster by one order of magnitude. Using asymptotically fast algorithms for division with remainder (Section 9.2) and Chinese remaindering (Section 10.3), the time even drops to $O(n\log^2 n \operatorname{loglog} n)$.

Notes. **5.1.** A general theory of these representations and conversions for polynomials and rational functions is in von zur Gathen (1986).

5.2. The Lagrange interpolant was invented in Waring (1779) and Lagrange (1795), page 286.

5.3. The secret sharing scheme is from Shamir (1979). Asmuth & Blakley (1982) propose using the CRA for fault-tolerant communication, and Rabin (1989) uses interpolation.

5.4. The name of the Chinese Remainder Theorem derives from the *Suan-ching* (arithmetic) of Sun-Tsŭ, written about the first century AD. He solves a particular problem (Exercise 5.15) in verse-form, using the integer versions of the "Lagrange interpolants" l_i as in the proof of Theorem 5.2; see Shen (1988) and Ku & Sun (1992). Variants of the question appear later in Chinese, Indian, and European mathematics, for example in Schwenter (1636), 3. Auffgab, with moduli 3, 5, 7, and 8. General solutions are due to Euler (1734/35a, 1747/48), Lagrange (1770b), §25, Gauß (1801), article 32, and Cauchy (1841).

Euler (1760/61) proved Corollary 5.6 about his totient function, and also that $a^k \equiv 1 \bmod m$ when $\gcd(a,m) = 1$ and $k = \operatorname{lcm}(\varphi(p_0^{e_0}),\ldots,\varphi(p_{r-1}^{e_{r-1}}))$, where $m = p_0^{e_0}\cdots p_{r-1}^{e_{r-1}}$ is the prime factorization of m (Exercise 18.13); this is also in Gauß (1801), article 92. Since Gauß (1801), article 38, the notation φ is used.

In a more general version of the Chinese Remainder Theorem, the moduli m_i are not required to be pairwise coprime, but only that $v_i \equiv v_j \bmod \gcd(m_i,m_j)$ for all i,j. Under these conditions solutions exist always and are unique modulo the least common multiple of all m_i (Exercise 5.23).

5.5. Gauß introduced his elimination method for astronomical calculations (Gauß 1809, article 182; Gauß 1810). Lagrange (1759) presented a similar procedure for 2×2 and

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

3×3 matrices. Edmonds (1967) and Bareiss (1968) showed that the intermediate results of Gaussian elimination over $\mathbb{Q}$ are polynomially bounded.

The modular determinant computation for polynomial matrices, called the interpolation method, is already in Mikeladze (1948); see also Faddeev & Faddeeva (1963), Section 49. Early suggestions for modular computer arithmetic are in Svoboda & Valach (1955), Svoboda (1957), and Garner (1959); see Szabó & Tanaka (1967) for a discussion.

5.8. Cauchy (1821) discusses his rational interpolation problem without paying attention to its solvability. Kronecker (1881a), page 544, was the first to point out the *bisher wohl noch nicht bemerkte Einschränkung der Lösbarkeit der* Cauchy*'schen Aufgabe*[1], namely, that (20) may have no solution (Exercise 5.36).

5.9. The Padé approximation problem derives its name from Padé's (1892) dissertation. It is a bit of a misnomer, since Kronecker (1881a) already stated and solved the problem, also proving the necessary and sufficient conditions for solvability (Corollary 5.21). However, Kronecker's approach was purely algebraic (as is ours), while Padé also considered functions like $\exp(x)$ and brought these approximations to the attention of numerical analysts. Jacobi (1846) had given an explicit solution to (23). Frobenius (1881) describes relations between the various Padé approximants of one power series.

Baker & Graves-Morris (1996) explain in detail the theory of Padé approximants, its connection with continued fractions, and its application to root and singularity finding, convergence acceleration, and various other problems in numerical analysis and theoretical physics. They also discuss the numerical stability of different methods for computing Padé approximants.

5.10. Theorem 5.26 is essentially in Kaltofen & Rolletschek (1989). The existence of a solution to (25) in the special case where $k = \lfloor \sqrt{m} \rfloor + 1$ was shown by Thue (1902).

5.11. The partial fraction decomposition is described in Euler (1748a), §39 ff. and Cauchy (1821), Chapter XI.

Exercises.

5.1 Let $m_0, \ldots, m_r \in \mathbb{N}_{\geq 2}$.

(i) Prove that every nonnegative integer $a < m_0 \cdots m_r$ has a **mixed-radix representation** of the form

$$a = a_0 + a_1 m_0 + a_2 m_0 m_1 \cdots + a_r m_0 \cdots m_{r-1},$$

with unique integers a_i satisfying $0 \leq a_i < m_i$ for all i. Relate this to the usual p-adic representation of an integer a, for an integer $p > 1$.

(ii) Compute the above representation of $a = 42$ for $m_0 = 2$, $m_1 = 3$, $m_2 = 2$, and $m_3 = 5$.

(iii) What is the analogous mixed-radix representation for polynomials?

5.2* Let $a = s/t \in \mathbb{Q}$, with coprime $s, t \in \mathbb{N}$ such that $0 < s < t$. With respect to an arbitrary base $p \in \mathbb{N}_{\geq 2}$, a has a unique periodic p-adic expansion

$$a = \sum_{i \geq 1} a_i p^{-i},$$

with all $a_i \in \{0, \ldots, p-1\}$. We say that this expansion is purely periodic if there is a positive $l \in \mathbb{N}$ such that $a_{i+l} = a_i$ for all $i \geq 1$, and the least such l is the length of the period. Moreover, we let $k \in \mathbb{N}$ be the smallest integer such that the sequence $a_{k+1}, a_{k+2}, \ldots$ is purely periodic, and call it the length of the preperiod. For example, the 10-adic representation of $1/6$ is $0.1\overline{6} = 1 \cdot 10^{-1} + \sum_{i \geq 2} 6 \cdot 10^{-i}$, with $k = l = 1$.

[1] constraint on the solvability of *Cauchy*'s problem, apparently unnoticed hitherto

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(i) Show that there exist unique $t^*, u \in \mathbb{N}$ with $t = ut^*$ such that $\gcd(p, t^*) = 1$ and every prime divisor of u divides p.

(ii) Prove that the p-adic expansion of a terminates (so that only finitely many a_i are nonzero) if and only if $t^* = 1$.

(iii) Show that the p-adic expansion of a is purely periodic if and only if p and t are coprime, and that then $l = \mathrm{ord}_t(p)$, the order of p in the multiplicative group $\mathbb{Z}_t^\times$.

(iv) Prove that $l = \mathrm{ord}_{t^*}(p)$ and $k = \min\{n \in \mathbb{N} : u \mid p^n\}$ in the general case.

(v) Conclude that $l \leq \varphi(t^*) < t$ and $k \leq \log_2 t$.

5.3 Let R be a ring (commutative, with 1) and $u \in R$. Prove that Horner's rule not only computes the remainder $f(u)$ of a polynomial $f \in R[x]$ of degree $n-1$ on division by $x-u$ but also the coefficients of the quotient $(f - f(u))/(x-u)$.

5.4 Let $\mathbb{F}_5 = \mathbb{Z}_5$ be the finite field with 5 elements.

(i) Compute a polynomial $f \in \mathbb{F}_5[x]$ of degree at most 2 satisfying

$$f(0) = 1, \quad f(1) = 2, \quad f(2) = 4 \tag{37}$$

(ii) List all polynomials $f \in \mathbb{F}_5[x]$ of degree at most 3 satisfying (37). How many of degree at most 4 are there? Generalize your answer to solutions of degree at most n for $n \in \mathbb{N}$.

5.5 Let $\mathbb{F}_7 = \mathbb{Z}_7$ be the finite field with 7 elements and $m = x(x+1)(x+6) = x^3 + 6x \in \mathbb{F}_7[x]$.

(i) Let $J \subseteq \mathbb{F}_7[x]$ be the set of all polynomials $h \in \mathbb{F}_7[x]$ solving the interpolation problem

$$h(0) = 1, \quad h(1) = 5, \quad h(6) = 2.$$

Compute the unique polynomial $f \in J$ of least degree.

(ii) Find a surjective ring homomorphism $\chi : \mathbb{F}_7[x] \longrightarrow \mathbb{F}_7^3$ such that $\ker \chi = \langle m \rangle = \{rm : r \in \mathbb{F}_7[x]\}$, and compute $\chi(f)$ and $\chi(x^2 + 3x + 2)$.

(iii) Show that $J = f + \ker \chi = \{f + rm : r \in \mathbb{F}_7[x]\}$.

5.6 Let $r = x^3 + x^2 \in \mathbb{F}_5[x]$.

(i) List all polynomials $f \in \mathbb{F}_5[x]$ of degree at most 5 satisfying

$$f(a) = r(a) \text{ for all } a \in \mathbb{F}_5. \tag{38}$$

(ii) How many polynomials $f \in \mathbb{F}_5[x]$ of degree at most 6 solve (38)?

5.7 (i) Show that $\sum_{0 \leq i < n} l_i = 1$, where l_i are the Lagrange interpolants as in (2).

(ii) Let $u_n \in F$ be another point different from $u_0, \ldots, u_{n-1}$. Show how one can obtain the Lagrange interpolants $l_0^*, \ldots, l_{n-1}^*, l_n^*$ corresponding to $u_0, \ldots, u_n$ from $l_0, \ldots, l_{n-1}$.

5.8 Let R be an integral domain, $u_0, \ldots, u_{n-1} \in R$, and $V = \mathrm{VDM}(u_0, \ldots, u_{n-1}) \in R^{n \times n}$. Prove that

$$\det V = \prod_{1 \leq j < i \leq n} (u_i - u_j).$$

Hint: Replace u_{n-1} by an indeterminate and proceed by induction on n.

5.9 Let F be a field, $u_0, \ldots, u_{n-1} \in F$ distinct, $m = (x-u_0)\cdots(x-u_{n-1}) \in F[x]$, and $f \in F[x]$. Prove that $fm' \equiv \sum_{0 \leq i < n} f(u_i) m/(x-u_i) \bmod m$, where m' is the formal derivative of m (Section 9.3). Hint: Exercise 9.22.

5.10 Let F be a field and $f/g \in F(x)$ a rational function, with $f, g \in F[x]$ such that $g \neq 0$ is monic and $\gcd(f, g) = 1$. We say that f/g is **defined** at a point $u \in F$ if $g(u) \neq 0$, and then the value of the rational function at u is $f(u)/g(u)$. Let $f^*/g^* \in F(x)$ be another rational function, with $g^* \neq 0$ monic and $\gcd(f^*, g^*) = 1$, such that f/g and f^*/g^* are defined and their values coincide at $n = \max\{\deg f + \deg g^*, \deg f^* + \deg g\} + 1$ distinct points $u_0, \ldots, u_{n-1} \in F$. Prove that $f = f^*$ and $g = g^*$.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

5.11* Another possibility to compute the interpolating polynomial of least degree is **Newton interpolation**. Suppose that $u_0,\ldots,u_{n-1},v_0,\ldots,v_{n-1}$ in a field F are given, with distinct $u_0,\ldots,u_{n-1}$, and let $f\in F[x]$ of degree less than n be the interpolating polynomial with $f(u_i)=v_i$ for all i. We divide f by $x-u_0$ with remainder and obtain $f=(x-u_0)g+f(u_0)=(x-u_0)g+v_0$ for some $g\in F[x]$ of degree $\deg f-1$. For $i\geq 1$, the value of g at u_i is $g(u_i)=(v_i-v_0)/(u_i-u_0)$, and we can determine g recursively in the same fashion.

(i) Design an algorithm for Newton interpolation, prove that is works correctly, and analyze its cost. It is possible to solve the problem with at most $\frac{5}{2}n^2$ operations in F.

(ii) Trace your algorithm on the examples of Exercises 5.4 and 5.5.

(iii) What is the connection to the mixed-radix representation of f, as discussed in Exercise 5.1, when $m_i=x-u_i$ for $0\leq i<n$?

5.12* Let F be a field, $u_0,\ldots,u_{n-1}\in F\setminus\{0\}$ with $u_i\neq\pm u_j$ for $0\leq i<j<n$, and $v_0,\ldots,v_{n-1}\in F$.

(i) Let $f\in F[x]$ of degree less than $2n$ be such that $f(u_i)=f(-u_i)$ for $0\leq i<n$. Prove that $f(x)=f(-x)$, so that f is **even**.

(ii) Use the Lagrange interpolation formula and (i) to show that there is a unique even interpolating polynomial $f\in F[x]$ of degree less than $2n$ such that $f(u_i)=v_i$ for $0\leq i<n$.

(iii) Let $g\in F[x]$ be the unique polynomial of degree less than n such that $g(u_i^2)=v_i$ for $0\leq i<n$. How is the polynomial f from (ii) related to g?

(iv) What are the statements corresponding to (i) through (iii) for odd interpolating polynomials, provided that $u_i\neq 0$ for all i?

(v) Compute an even polynomial $f_0\in\mathbb{R}[x]$ of degree at most 4 interpolating the cosine function at $u_0=\pi/6$, $u_1=\pi/3$, and $u_2=\pi/2$, and an odd polynomial $f_1\in\mathbb{R}[x]$ of degree at most 5 interpolating the sine function at those points.

(Euler (1783) stated interpolation formulas for odd and even functions.)

5.13* In this exercise, we discuss bivariate interpolation.

(i) Develop an algorithm for computing $f\in F[x,y]$, F a field, where the degree of f in y is less than n and

$$f(x,u_i)=v_i \text{ for } i=0,1,\ldots,n-1,$$

for distinct $u_i\in F$ and arbitrary $v_i\in F[x]$. Show that f is unique.

(ii) Assuming that the degree of each v_i is less than m, what is the computing time of your algorithm (in terms of m and n)?

(iii) Compute $f\in\mathbb{F}_{11}[x,y]$ such that

$$f(x,0)=x^2+7,\quad f(x,1)=x^3+2x+3,\quad f(x,2)=x^3+5.$$

5.14 Let F be a field, $f\in F[x]$ of degree less than n, and $u_0,\ldots,u_{n-1}\in F\setminus\{0\}$ distinct. Determine the set of all interpolation polynomials $g\in F[x]$ of degree less than n with $g(u_i)=f(u_i)$ for $0\leq i\leq n-2$. (In the situation of Section 5.3, this represents the knowledge of all players minus player $n-1$.) Let $c\in F$. How many of these g have constant coefficient c? (Your answer should imply that the secret sharing scheme is secure.)

5.15 What is the least nonnegative integer f with $f\equiv 2 \bmod 3$, $f\equiv 3 \bmod 5$, and $f\equiv 2 \bmod 7$?

5.16 How many common solutions $f\in\mathbb{Z}$ with $0\leq f<10^6$ do the following congruences possess?

$$f\equiv 2 \bmod 11,\quad f\equiv -1 \bmod 13,\quad f\equiv 10 \bmod 17.$$

5.17 Carl Friedrich, Joachim, and Jürgen met at a Sylvester party on Thursday, 31 December 1998. They agreed to play Skat (a German card game) together some day as soon as all of them find the time to do so. But they got into the usual troubles: Carl Friedrich was busy except on Fridays, Joachim had time on 7 January and then again every 9th day, and Jürgen was free on 6 January and then again every 11th day. Which date did they agree upon?

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

5.18 Ernie, Bert, and the Cookie Monster want to measure the length of Sesame Street. Each of them does it his own way. Ernie relates: "I made a chalk mark at the beginning of the street and then again every 7 feet. There were 2 feet between the last mark and the end of the street." Bert tells you: "Every 11 feet, there are lamp posts in the street. The first one is 5 feet from the beginning, and the last one is exactly at the end of the street." Finally, the Cookie Monster says: "Starting at the beginning of Sesame Street, I put down a cookie every 13 feet. I ran out of cookies 22 feet from the end." All three agree that the length does not exceed 1000 feet. How long is Sesame Street?

5.19 (i) Find a polynomial in $\mathbb{F}_5[x]$ of degree four which is reducible but has no roots in $\mathbb{F}_5$. Are there such examples of lower degree?

(ii) Which of the following polynomials in $\mathbb{F}_5[x]$ are irreducible, which are reducible?

$$m_0 = x^2+2, \quad m_1 = x^2+3x+4, \quad m_2 = x^3+2, \quad m_3 = x^3+x+1.$$

(iii) Conclude that the system

$$f \equiv x+1 \bmod m_0, \quad f \equiv 3 \bmod m_1$$

has a solution $f \in \mathbb{F}_5[x]$, and compute the unique solution of least degree.

5.20 Compute a solution $f \in \mathbb{F}_5[x]$ of the system of congruences

$$f \equiv 1 \mod x+1, \qquad x \cdot f \equiv x+1 \mod x^2+1, \qquad (x+1)f \equiv x+1 \mod x^3+1$$

such that $\deg f < 5$. Hint: First bring each of the congruences into the form $f \equiv v \bmod m$ for some $v, m \in \mathbb{F}_5[x]$, using Exercise 4.15. What is the set of all solutions without the degree constraint?

5.21$^{\longrightarrow}$ Let $m_0 = x^2+1$, $m_1 = x^2-1$, $m_2 = x^3+x-1$, $v_0 = -x$, $v_1 = x+1$, and $v_2 = x^5-x$ in $\mathbb{F}_3[x]$.

(i) How many polynomials $f \in \mathbb{F}_3[x]$ are there with $f \equiv v_i \bmod m_i$ for $i = 0,1,2$, and $\deg f \leq 8$? Answer this *without* solving (ii).

(ii) Give a list of all f as in (i).

5.22* Let $p_0, p_1 \in \mathbb{N}$ be distinct primes, $m = p_0 p_1$, $n \in \mathbb{N}$, and $u_0, \ldots, u_{n-1}, v_0, \ldots, v_{n-1} \in \mathbb{Z}$.

(i) Show that there exists an interpolating polynomial $f \in \mathbb{Z}[x]$ such that

$$f \text{ has coefficients in } \{0, \ldots, m-1\}, \deg f < n, \text{ and } f(u_i) \equiv v_i \bmod m \text{ for } 0 \leq i < n \qquad (39)$$

if and only if

$$u_i \equiv u_j \bmod p_k \Longrightarrow v_i \equiv v_j \bmod p_k$$

for $0 \leq i < j < n$ and $k = 0, 1$.

(ii) Show that (39) has a unique solution if and only if $u_i \not\equiv u_j \bmod p_k$ for $0 \leq i < j < n$ and $k = 0, 1$.

(iii) Compute all interpolating polynomials $f \in \mathbb{Z}[x]$ with coefficients in $\{0, \ldots, 14\}$, $\deg f < 3$, and

$$f(1) \equiv 2 \bmod 15, \; f(2) \equiv 5 \bmod 15, \; f(4) \equiv -1 \bmod 15.$$

5.23* (i) Let R be a Euclidean domain, $m_0, m_1 \in R \setminus \{0\}$, and $v_0, v_1 \in R$. Show that

$$f \equiv v_0 \mod m_0, \quad f \equiv v_1 \mod m_1$$

has a solution $f \in R$ if and only if $v_0 \equiv v_1 \bmod \gcd(m_0, m_1)$. Hint: Theorem 4.10.

(ii) Compute one particular solution for $R = \mathbb{Z}$, $m_0 = 36$, $m_1 = 42$, $v_0 = 2$, $v_1 = 8$, and describe the set of all solutions.

5.24* Design and analyze a Chinese Remainder Algorithm à la "Newton interpolation" (Exercise 5.11). Trace your algorithm on the problems of Exercises 5.16 and 5.20.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

5.25 Are the two rings $\mathbb{Z}_5 \times \mathbb{Z}_{12}$ and $\mathbb{Z}_3 \times \mathbb{Z}_{20}$ isomorphic?

5.26 Enumerate $\mathbb{Z}_m^\times$ and determine $\varphi(m)$ for $m = 11, 16, 33, 42$.

5.27 Make a list showing all integers m for which $\varphi(m) \leq 10$, and prove that your list is complete.

5.28 Let $\mathbb{F}_q$ be a finite field with q elements and $f = f_0^{e_0} \cdots f_{r-1}^{e_{r-1}}$ with $f_0, \ldots, f_{r-1} \in \mathbb{F}_q[x]$ irreducible and pairwise coprime and $e_0, \ldots, e_{r-1} \in \mathbb{N}_{>0}$. Let $n = \deg f$ and $n_i = \deg f_i$ for all i. Recall the analog Φ of Euler's totient function (Exercise 4.19). Prove that

$$\Phi(f) = (q^{n_0} - 1)q^{n_0(e_0-1)} \cdots (q^{n_{r-1}} - 1)q^{n_{r-1}(e_{r-1}-1)} = q^n \prod_{0 \leq i < r} \left(1 - \frac{1}{q^{n_i}}\right).$$

Hint: CRT.

5.29* Prove Theorem 5.8.

5.30 Let $A_n = (i^j)_{1 \leq i,j \leq n} \in \mathbb{Z}^{n \times n}$.

(i) Compute a good upper bound on $|\det A_n|$ in terms of n using Hadamard's inequality 16.6.

(ii) Compute $\det A_3$ with the small primes modular algorithm.

5.31 Use the familiar formula $\det A = \sum_{\sigma \in S_n} \operatorname{sign}(\sigma) \cdot a_{1\sigma(1)} \cdots a_{n\sigma(n)}$ for the determinant of a square matrix $A \in \mathbb{Z}^{n \times n}$, where S_n is the symmetric group of all $n!$ permutations of $\{1, \ldots, n\}$ (Section 25.1), to derive an upper bound on $|\det A|$ in terms of n and $B = \max_{1 \leq i,j \leq n} |a_{ij}|$. Compare this to the Hadamard bound, and tabulate both bounds and their ratio for $1 \leq n \leq 10$.

5.32$\longrightarrow$ Let F be a field, $n \in \mathbb{N}_{>0}$, and $A = (a_{ij})_{1 \leq i,j \leq n} \in F[x]^{n \times n}$ a square matrix with polynomial entries. Moreover, let $m = \max\{\deg a_{ij} : 1 \leq i, j \leq n\}$.

(i) Find a tight upper bound $r \in \mathbb{N}$ on $\deg(\det A)$ in terms of m and n.

(ii) Describe an algorithm for computing $\det A$ using a small primes modular approach if the field F has more than r elements. Hint: Choose linear moduli. How many operations in F does your algorithm use (in terms of n and m)?

(iii) Use your algorithm to compute the determinant of the matrix

$$A = \begin{pmatrix} -x+1 & 0 & 2 \\ x & x+1 & 2x \\ 2x & 3x+1 & x \end{pmatrix} \in \mathbb{F}_7[x]^{3 \times 3}.$$

(iv) Find a tight upper bound on $\deg(\det A)$ in terms of the maximal degrees m_i in the ith row of A for $1 \leq i \leq n$. (Sometimes, this bound or the corresponding bound arising from the maximal column degrees is better than the bound from (i).)

(v) Using the bound from (iv), compute the determinant of

$$A = \begin{pmatrix} x-1 & x-2 & x-3 \\ 2x+1 & 2x+3 & 2x-2 \\ x^2-1 & x^2+x+1 & (x-1)^2 \end{pmatrix} \in \mathbb{F}_7[x]^{3 \times 3}.$$

5.33* The goal of this exercise is to show that nonsingular linear systems over $\mathbb{Q}$ can be solved in polynomial time using a modular approach. Thus let $A \in \mathbb{Z}^{n \times n}$ and $b \in \mathbb{Z}^n$ for some $n \in \mathbb{N}$, and assume that $\det A \neq 0$. Then there is a unique solution $x \in \mathbb{Q}^n$ of the linear system $Ax = b$, namely $x = A^{-1}b$.

(i) Given a bound $B \in \mathbb{N}$ on the absolute values of the entries of A and b, show that the numerators and denominators of the coefficients of x are less than $n^{n/2}B^n$ in absolute value. Hint: Use Cramer's rule 25.6 and Hadamard's inequality 16.6.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(ii) We consider the following modular algorithm. Choose a prime $p \in \mathbb{N}$ greater than $2n^n B^{2n}$, and perform Gaussian elimination on A mod p and b mod p. Convince yourself that $p \nmid \det A$. Find a $y \in \mathbb{Z}^n$ such that y mod p is the unique solution of the modular linear system $(A \bmod p)(y \bmod p) = b \bmod p$. Now $x \equiv y \bmod p$, and we can reconstruct x from y using rational number reconstruction (Section 5.10) for each of the coefficients. Prove that this algorithm works correctly.

(iii) Show that the running time of the algorithm is $O(n^3 \log^2 p)$ word operations, or $O^\sim(n^5 \log^2 B)$ when p is close to $2n^n B^{2n}$.

(iv) Run your algorithm on the matrix A_3 from Example 5.30 and the vector $b = (1,1,1)^T$.

5.34* Given are a positive integer $n \in \mathbb{N}$, two polynomials $a = \sum_{0 \le i < n} a_i x^i$, $b = \sum_{0 \le i < n} b_i x^i$ in $\mathbb{Z}[x]$, and a bound $B \in \mathbb{N}$ on the coefficients such that $|a_i|, |b_i| \le B$ for $0 \le i < n$. Moreover, let $ab = c = \sum_{0 \le i < 2n} c_i x^i \in \mathbb{Z}[x]$.

(i) Find a tight common upper bound on the $|c_i|$ in terms of n and B.

(ii) Describe an algorithm for the computation of c using a small primes modular approach.

(iii) Trace your algorithm on the computation of the product of

$$a = 987x^3 + 654x^2 + 321x, \quad b = -753x^3 - 333x^2 - 202x + 815.$$

5.35⟶ Let $n+1$ points $u_0 < u_1 < \cdots < u_n$ in $\mathbb{R}$ be given, and $v_0, \ldots, v_n \in \mathbb{R}$ arbitrary. You are to find a twice continuously differentiable function $f: [u_0, u_n] \longrightarrow \mathbb{R}$ which takes the value v_i at point u_i and has $f'(u_0) = f''(u_0) = 0$, as follows. Construct a sequence of polynomials $f_0, \ldots, f_n \in \mathbb{R}[x]$ of degree at most 3 such that $f_0 = v_0$,

$$f_i(u_{i-1}) = f_{i-1}(u_{i-1}),\ f_i'(u_{i-1}) = f_{i-1}'(u_{i-1}),\ f_i''(u_{i-1}) = f_{i-1}''(u_{i-1}),\ f_i(u_i) = v_i,$$

for $1 \le i \le n$. This amounts to solving a Hermite interpolation problem for each interval $[u_{i-1}, u_i]$, with three conditions on f_i and its first two derivatives at the left boundary and one condition on f_i at the right boundary. Then f is defined to be equal to f_i on the interval $[u_{i-1}, u_i]$, for all i. Such an f is called a **cubic spline**.

(i) Prove that $f_0, \ldots, f_n$ exist uniquely.

(ii) Compute and draw the cubic spline for the data $u_i = i$ for $0 \le i \le 3$, $v_0 = v_3 = 1$, $v_2 = 0$, and $v_3 = -1$.

(iii) In (ii), give various other values to v_2, say $-5, -3, -2, -1, 1, 2, 3, 5, 7$.

5.36 (Kronecker 1881a, page 546) Let $F = \mathbb{Q}$, $n = 4$, $u_i = i + 1$ for $i = 0, \ldots, 3$, $v_0 = 6$, $v_1 = 3$, $v_2 = 2$, $v_3 = 3$. Show that for $k = 2$, (21) has a unique solution $r, t \in \mathbb{Q}[x]$ with t monic, while (20) is unsolvable.

5.37 For $1 \le k \le 5$, try to solve the Cauchy interpolation problem

$$t(i) \neq 0 \text{ and } \frac{r}{t}(i) = v_i \text{ for } 0 \le i \le 4, \quad \gcd(r,t) = 1 \tag{40}$$

for polynomials $r, t \in \mathbb{F}_5[x]$ with $\deg r < k$ and $\deg t \le 5 - k$, where the v_i are given by the following table.

i	0	1	2	3	4
v_i	1	2	3	2	1

For which values of k is there no solution?

5.38 Let F be a field, $u_0, \ldots, u_{n-1} \in F$ distinct, $v_0, \ldots, v_{n-1} \in F$, and $S = \{0 \le i < n : v_i = 0\}$. Show that the Cauchy interpolation problem (20) has no solution if $k \le \#S < n$.

5.39 Tabulate all $(k, n-k)$-Padé approximants to $g = x^4 + x^3 + 3x^2 + 1 \in \mathbb{F}_5[x]$ for $0 \le k \le n \le 5$. Mark the entries in the table where no approximant exists.

5.40⟶ Give all Padé approximants in $\mathbb{Q}(x)$ to the exponential function $\exp(x) = 1 + x + x^2/2 + x^3/6 + x^4/24 + \cdots$ modulo x^5.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

5.41 Let F be a field, $n \in \mathbb{N}$, $g \in F[x]$ of degree less than n, and $\ell \in \mathbb{N}_{>0}$ the number of division steps in the Euclidean Algorithm for the pair (x^n, g).

(i) Show that there are at most ℓ distinct coprime pairs $(r,t) \in F[x]$ such that $t \neq 0$ is monic,

$$r \equiv tg \bmod x^n \text{ and } \deg r + \deg t < n.$$

Thus there are at most ℓ distinct Padé approximants to g modulo x^n.

(ii) Given the jth row $r_j = s_j x^n + t_j g$ in the Extended Euclidean Algorithm, for some $j \in \{1,\ldots,\ell\}$, and the degree sequence $n_0 = n, n_1, \ldots, n_\ell$ of (x^n, g), for which values of $k \in \{1,\ldots,n\}$ is (r_j, t_j) a solution of (23)?

5.42* This exercise discusses both Cauchy interpolation and Padé approximation from a more general point of view. We let $m_0, \ldots, m_{l-1} \in F[x]$ be nonconstant, monic, and pairwise coprime, $m = m_0 \cdots m_{l-1}$ of degree n, $v_0, \ldots, v_{l-1} \in F[x]$ such that $\deg v_i < \deg m_i$ for all i, and $0 \leq k \leq n$. (In contrast to Section 5.4, we denote the number of moduli by l.) The **rational Chinese Remainder Problem** is to compute polynomials $r,t \in F[x]$ satisfying

$$\gcd(t, m_i) = 1 \text{ and } rt^{-1} \equiv v_i \bmod m_i \text{ for } 0 \leq i < l, \deg r < k, \text{ and } \deg t \leq n-k, \tag{41}$$

where t^{-1} is the modular inverse of t modulo m_i (Section 4.2). Let $g \in F[x]$ be the polynomial solution of the system of congruences $g \equiv v_i \bmod m_i$ for all i. Furthermore, let $r_j, s_j, t_j \in F[x]$ be the jth row in the Extended Euclidean Algorithm for m and g, where j is minimal such that $\deg r_j < k$. Prove:

(i) There exist polynomials $r,t \in F[x]$ satisfying

$$r \equiv tv_i \bmod m_i \text{ for } 0 \leq i < l, \quad \deg r < k, \quad \deg t \leq n-k, \tag{42}$$

namely $r = r_j$ and $t = t_j$. If in addition $\gcd(r_j, t_j) = 1$, then r and t also solve (41).

(ii) If $r/t \in F(x)$ is a canonical form solution to (41), then $r = \tau^{-1} r_j$ and $t = \tau^{-1} t_j$, where $\tau = \mathrm{lc}(t_j) \in F \setminus \{0\}$. In particular, (41) is solvable if and only if $\gcd(r_j, t_j) = 1$.

5.43 Compute—if possible—rational functions $\rho = r/t \in \mathbb{Q}(x)$ satisfying

(i) $\rho(-1) = 1$, $\rho(0) = 2$, $\rho(1) = 1$, $\rho'(1) = -1$, with $\deg r < 3$, $\deg t \leq 1$.

(ii) $\rho(-1) = 2$, $\rho'(-1) = 1$, $\rho(1) = -1$, $\rho'(1) = 2$, with $\deg r < 1$, $\deg t \leq 3$.

5.44* Use Theorem 5.26 to formulate and prove the analog of Exercise 5.42 for integers.

5.45 Compute the partial fraction expansions of the following rational functions over $\mathbb{Q}$:

(i) $\dfrac{x+2}{(x+1)^3(x-1)^2}$, (ii) $\dfrac{x^4+2x-1}{x^3(x^2+1)}$.

5.46 A cubic **Bézier curve** is a parametric curve in $\mathbb{R}^2$ of the form

$$A \cdot (i+1-u)^3 + B \cdot 3(i+1-u)^2(u-i) + C \cdot 3(i+1-u)(u-i)^2 + D \cdot (u-i)^3,$$

where A, B, C, D are points in $\mathbb{R}^2$, $i \in \mathbb{N}$, and the parameter u runs through the real interval $[i, i+1]$. (These curves, and also similar surfaces, were introduced in the late 1960s by Bézier at the Renault car company, and by de Casteljau at Citroën; see Bézier (1970) and de Casteljau (1985).) A cubic **Bézier spline** is a parametric curve on the real interval $[0,\ldots,n]$ which is defined by

$$A_i \cdot (i+1-u)^3 + B_i \cdot 3(i+1-u)^2(u-i) + C_i \cdot 3(i+1-u)(u-i)^2 + A_{i+1} \cdot (u-i)^3$$

for $u \in [i, i+1]$ and $0 \leq i < n$, i.e., it is a Bézier curve on each interval $[i, i+1]$. The points $A_0, B_0, C_0, A_1, \ldots, A_{n-1}, B_{n-1}, C_{n-1}, A_n \in R^2$ are the **control points** of the Bézier spline.

(i) Show that the Bézier spline is continuous and passes through the points $A_0, \ldots, A_n$.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

(ii) Prove that the Bézier spline is continuously differentiable with respect to the parameter u if and only if $B_i - A_i = A_i - C_{i-1}$ for $1 \leq i < n$. Show that in this case, $B_i - A_i$ is the tangent vector to the curve at the point A_i for $0 \leq i < n$. What is the tangent vector at the point A_n?

(iii) Consider the following control points for $n = 4$:

i	0	1	2	3	4
A_i	$(-1.8, 0)$	$(-3.8, 8.5)$	$(0, 12.8)$	$(3.8, 8.5)$	$(1.8, 0)$
B_i	$(-1.8, 2)$	$(-3.8, 11.17)$	$(2, 12.8)$	$(3.8, 5.83)$	
C_i	$(-3.8, 5.83)$	$(-2, 12.8)$	$(3.8, 11.17)$	$(1.8, 2)$	

The corresponding Bézier spline models the inner boundary of the uppercase greek letter Ω. Plot this Bézier spline. Is it continuously differentiable? Is it twice continuously differentiable?

(iv) Prove that for arbitrary points $A_0, \ldots, A_n \in \mathbb{R}^2$, there exists a Bézier spline passing through these points that is twice continuously differentiable with respect to the parameter u. Show that there is a unique such spline if B_0 and C_0 are prescribed as well. Hint: Exercise 5.35.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press

The mathematician's pattern, like a painter's or the poet's, must be *beautiful.* [. . .] Beauty is the first test; there is no permanent place in the world for ugly mathematics.

Godfrey Harold Hardy (1940)

Zudem ist es ein Irrtum zu glauben, daß die Strenge in der Beweisführung die Feindin der Einfachheit sei. An zahlreichen Beispielen finden wir im Gegenteil bestätigt, daß die strenge Methode auch zugleich die einfachere und leichter faßliche ist. Das Streben nach Strenge zwingt uns eben zur Auffindung einfacherer Schlußweisen.[1]

David Hilbert (1900)

Der Mathematiker ist nur in sofern vollkommen, als er ein vollkommener Mensch ist, als er das Schöne des Wahren in sich empfindet; dann erst wird er gründlich, durchsichtig, umsichtig, rein, klar, anmutig, ja elegant wirken.[2]

Johann Wolfgang von Goethe (1829)

The algebraical element looked to me a pure science, subject to mathematical law, inhuman.

Thomas Edward Lawrence (1926)

[1] Furthermore, it is an error to believe that rigor in proof is an enemy of simplicity. On the contrary we find it confirmed by numerous examples that the rigorous method is at the same time the simpler and the more comprehensible one. The very effort for rigor forces us to find simpler proof methods.

[2] The mathematician is perfect only in so far as he is a perfect being, in so far as he perceives the beauty of truth; only then will he appear to be thorough, transparent, comprehensive, pure, clear, gracious, and even elegant.

https://doi.org/10.1017/CBO9781139856065.007 Published online by Cambridge University Press